

Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA DE  
PLATAFORMA MULTIAGENTE AUTÔNOMA  
PARA PRODUÇÃO DE CONHECIMENTO  
CIENTÍFICO VERIFICÁVEL**

Fortaleza – Ceará

2026



Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA DE  
PLATAFORMA MULTIAGENTE AUTÔNOMA PARA  
PRODUÇÃO DE CONHECIMENTO CIENTÍFICO  
VERIFICÁVEL**

Tese apresentada ao Programa de Pós-Graduação em Engenharia de Software da Universidade Federal do Ceará como requisito parcial para obtenção do título de Doutor em Engenharia de Software.

Universidade Federal do Ceará

Orientador: [ORIENTADOR]  
Coorientador: [COORIENTADOR]

Fortaleza – Ceará  
2026



### **Ficha Catalográfica**

LARANJEIRA, Marcelo Claro

OpenCode Ecosystem: Arquitetura de Plataforma Multiagente Autônoma para  
Produção de Conhecimento Científico Verificável.

Fortaleza, 2026.

62 p.

Tese (Doutorado em Engenharia de Software) –  
Universidade Federal do Ceará.

1. Engenharia de Software. 2. Sistemas Multiagente. 3. Model Context  
Protocol.  
4. Verificação Formal. 5. Ecossistema OpenCode. 6. Arquitetura de Software. I.  
Título.



# Agradecimentos

Ao Programa de Pós-Graduação em Engenharia de Software da Universidade Federal do Ceará pelo ambiente de excelência acadêmica e suporte institucional.

À comunidade open-source internacional, cujos projetos MiroFish (666ghj/MiroFish), BetaFish (666ghj/BettaFish), DeerFlow (bytedance/deer-flow) e OpenCode CLI (anomalyco/opencode) forneceram os fundamentos arquiteturais que inspiraram esta pesquisa.

Aos revisores anônimos cujas críticas substantivas aprimoraram significativamente a qualidade deste trabalho.

Aos 125 agentes do ecossistema OpenCode, cuja operação contínua demonstra a viabilidade de sistemas multiagente autônomos para engenharia de software e produção de conhecimento científico verificável.

*“The fundamental problem of software engineering is  
not the production of programs, but the production  
of knowledge about programs.”*

— David Lorge Parnas



## RESUMO

Esta tese investiga o Ecossistema OpenCode como plataforma multiagente autônoma para engenharia de software e produção de conhecimento científico verificável. A pesquisa aborda o problema fundamental de como arquitetar sistemas onde agentes de inteligência artificial operam colaborativamente mantendo verificabilidade formal, rastreabilidade integral e qualidade de engenharia. A plataforma integra 125 agentes especializados, 46 Model Context Protocols (MCPs), 150 skills em 13 categorias, 15 plugins TypeScript e 14 comandos, todos orquestrados por um pipeline de verificação formal Cora-Debate com 7 verificadores simbólicos (V1-V7). A arquitetura de três camadas (MCP→Skill→Agent), inspirada em padrões de engenharia de software como hexagonal architecture, event-driven design e CQRS, permite desacoplamento entre componentes e evolução independente de cada camada. A pesquisa documenta 13 ciclos evolutivos (score 85→100) através de metodologia mista combinando estudo de caso longitudinal, pesquisa-ação e análise quantitativa automatizada. O motor de raciocínio multi-paradigma integra 212 tipos de raciocínio em 27 categorias, com quatro engines formais complementares: Z3 Theorem Prover para verificação SMT, SymPy para manipulação simbólica, miniKanren para programação lógica relacional, e Critical Engine para detecção de 15 falácias. O pipeline SDD+TDD acadêmico, validado por 200 casos de teste em 9 especificações técnicas, demonstra que a integração de especificação formal com desenvolvimento dirigido por testes produz software de qualidade verificável. A auditoria caixa branca com protocolo TSAC, logging imutável JSONL e rastreamento SHA-256 garante integridade de cada afirmação. Os resultados demonstram Health Score 100/100, 100% de aprovação nos testes Cora-Debate, e trajetória de melhoria contínua de +17.6%. As contribuições incluem: (i) arquitetura de referência MCP-first para ecossistemas multiagente; (ii) taxonomia de 212 tipos de raciocínio com validação empírica; (iii) framework de verificação formal multi-paradigma integrando SMT, álgebra simbólica e lógica relacional; (iv) metodologia SDD+TDD para engenharia de software com agentes de IA; (v) sistema de auditoria acadêmica caixa branca com rastreabilidade criptográfica.

**Palavras-chave:** Engenharia de Software. Sistemas Multiagente. Model Context Protocol. Verificação Formal. Arquitetura de Software. Ecossistema OpenCode.

## ABSTRACT

This thesis investigates the OpenCode Ecosystem as an autonomous multi-agent platform for software engineering and verifiable scientific knowledge production. The research addresses the fundamental problem of architecting systems where AI agents operate collaboratively while maintaining formal verifiability, complete traceability, and engineering quality. The platform integrates 125 specialized agents, 46 Model Context Protocols (MCPs), 150 skills across 13 categories, 15 TypeScript plugins, and 14 commands, all orchestrated by a Cora-Debate formal verification pipeline with 7 symbolic verifiers (V1-V7). The three-layer architecture (MCP→Skill→Agent), inspired by software engineering patterns such as hexagonal architecture, event-driven design, and CQRS, enables component decoupling and independent layer evolution. The research documents 13 evolutionary cycles (score 85→100) through mixed methodology combining longitudinal case study, action research, and automated quantitative analysis. The multi-paradigm reasoning engine integrates 212 reasoning types across 27 categories, with four complementary formal engines: Z3 Theorem Prover for SMT verification, SymPy for symbolic manipulation, miniKanren for relational logic programming, and Critical Engine for detection of 15 fallacies. The academic SDD+TDD pipeline, validated by 200 test cases across 9 technical specifications, demonstrates that integrating formal specification with test-driven development produces verifiable quality software. White-box auditing with TSAC protocol, immutable JSONL logging, and SHA-256 tracking ensures integrity of every claim. Results demonstrate Health Score 100/100, 100% approval in Cora-Debate tests, and continuous improvement trajectory of +17.6%. Contributions include: (i) MCP-first reference architecture for multi-agent ecosystems; (ii) taxonomy of 212 reasoning types with empirical validation; (iii) multi-paradigm formal verification framework integrating SMT, symbolic algebra, and relational logic; (iv) SDD+TDD methodology for AI-agent software engineering; (v) white-box academic auditing system with cryptographic traceability.

**Keywords:** Software Engineering. Multi-Agent Systems. Model Context Protocol. Formal Verification. Software Architecture. OpenCode Ecosystem.

# Lista de ilustrações

## Lista de tabelas

|                                                                                                        |    |
|--------------------------------------------------------------------------------------------------------|----|
| Tabela 1 – Analogias entre ecossistemas biológicos e o OpenCode . . . . .                              | 27 |
| Tabela 2 – Critérios de inclusão e exclusão . . . . .                                                  | 29 |
| Tabela 3 – Motivações para LLM-MAS em geração de código (adaptado de (RASHEED et al., 2026)) . . . . . | 30 |
| Tabela 4 – Comparativo de abordagens de verificação de código LLM . . . . .                            | 30 |
| Tabela 5 – Lacunas na literatura e contribuições desta tese . . . . .                                  | 31 |
| Tabela 6 – Catálogo de servidores MCP por categoria funcional . . . . .                                | 34 |
| Tabela 7 – Distribuição completa das 150 skills . . . . .                                              | 36 |
| Tabela 8 – Tipologia dos 125 agentes . . . . .                                                         | 37 |
| Tabela 9 – Taxonomia completa de raciocínios v12.0 . . . . .                                           | 39 |
| Tabela 10 – Resultados CORA-Eval Benchmark (200 casos de teste) . . . . .                              | 43 |
| Tabela 11 – Métricas de qualidade de código . . . . .                                                  | 46 |
| Tabela 12 – Detalhamento dos 13 ciclos evolutivos . . . . .                                            | 47 |
| Tabela 13 – Critérios de avaliação Qualis A1 . . . . .                                                 | 50 |
| Tabela 14 – Métricas consolidadas do Ecossistema OpenCode . . . . .                                    | 53 |
| Tabela 15 – Desempenho individual dos verificadores V1-V7 . . . . .                                    | 54 |

# Sumário

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Agradecimentos</b>                                       | <b>5</b>  |
| <b>Lista de ilustrações</b>                                 | <b>9</b>  |
| <b>Lista de tabelas</b>                                     | <b>9</b>  |
| <b>Sumário</b>                                              | <b>10</b> |
| <b>1 Introdução</b>                                         | <b>15</b> |
| 1.1 Contextualização e Motivação                            | 15        |
| 1.2 Formulação do Problema                                  | 15        |
| 1.3 Hipóteses de Pesquisa                                   | 16        |
| 1.4 Objetivos                                               | 17        |
| 1.4.1 Objetivo Geral                                        | 17        |
| 1.4.2 Objetivos Específicos                                 | 17        |
| 1.5 Delimitação do Escopo                                   | 17        |
| 1.6 Contribuições Esperadas                                 | 18        |
| 1.7 Estrutura da Tese                                       | 18        |
| <b>2 Fundamentação Teórica</b>                              | <b>19</b> |
| 2.1 Sistemas Multiagente: Fundamentos e Evolução            | 19        |
| 2.1.1 Definições Formais                                    | 19        |
| 2.1.2 Evolução Histórica dos Paradigmas MAS                 | 19        |
| 2.1.3 Arquiteturas de Agentes Baseados em LLM               | 20        |
| 2.1.4 Desafios Fundamentais em LLM-MAS                      | 20        |
| 2.2 Model Context Protocol: Arquitetura e Fundamentos       | 21        |
| 2.2.1 Especificação do Protocolo                            | 21        |
| 2.2.2 Topologia de Comunicação                              | 21        |
| 2.2.3 Propriedades Formais da Arquitetura MCP               | 22        |
| 2.2.4 MCP como Padrão de Engenharia de Software             | 22        |
| 2.2.5 Limitações e Desafios do MCP                          | 22        |
| 2.3 Engenharia de Software com Agentes Inteligentes         | 23        |
| 2.3.1 Spec-Driven Development (SDD): Teoria e Prática       | 23        |
| 2.3.2 Test-Driven Development (TDD): Formalização           | 23        |
| 2.3.3 Integração SDD+TDD: O Pipeline OpenCode               | 24        |
| 2.3.4 Padrões de Projeto para Agentes de IA                 | 24        |
| 2.4 Motores de Raciocínio Formal                            | 25        |
| 2.4.1 Fundamentos de satisfatibilidade Modulo Teorias (SMT) | 25        |
| 2.4.2 Álgebra Simbólica com SymPy                           | 25        |
| 2.4.3 Programação Lógica Relacional com miniKanren          | 25        |
| 2.4.4 Critical Engine: Detecção de Falácias e Vieses        | 26        |

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| 2.4.5    | Integração dos Quatro Motores . . . . .                                         | 26        |
| 2.5      | Ecosistemas de Software Evolutivos . . . . .                                    | 26        |
| 2.5.1    | Fundamentos de Evolução de Software . . . . .                                   | 26        |
| 2.5.2    | Ciclo PLAN-ACT-REFLECT-EXTRACT-EVOLVE . . . . .                                 | 27        |
| 2.5.3    | Métricas de Evolutibilidade . . . . .                                           | 27        |
| 2.5.4    | Comparação com Ecosistemas Biológicos . . . . .                                 | 27        |
| <b>3</b> | <b>Estado da Arte e Revisão Sistemática . . . . .</b>                           | <b>29</b> |
| 3.1      | Metodologia da Revisão . . . . .                                                | 29        |
| 3.1.1    | CrITÉRIOS de Inclusão e Exclusão . . . . .                                      | 29        |
| 3.2      | Análise Temática da Literatura . . . . .                                        | 29        |
| 3.2.1    | Eixo 1: Sistemas Multiagente Baseados em LLM para Código . . . . .              | 29        |
| 3.2.2    | Eixo 2: Model Context Protocol . . . . .                                        | 30        |
| 3.2.3    | Eixo 3: Verificação Formal e Qualidade de Código . . . . .                      | 30        |
| 3.2.4    | Eixo 4: Integridade Acadêmica e Detecção de IA . . . . .                        | 31        |
| 3.2.5    | Eixo 5: Arquitetura de Software e Conhecimento . . . . .                        | 31        |
| 3.3      | Síntese Crítica e Lacunas Identificadas . . . . .                               | 31        |
| <b>4</b> | <b>Arquitetura do Ecosistema OpenCode . . . . .</b>                             | <b>33</b> |
| 4.1      | Visão Arquitetural . . . . .                                                    | 33        |
| 4.1.1    | Princípios Arquiteturais . . . . .                                              | 33        |
| 4.2      | Camada de Infraestrutura ( $\mathcal{L}_1$ ): Model Context Protocols . . . . . | 34        |
| 4.2.1    | Topologia de Servidores MCP . . . . .                                           | 34        |
| 4.2.2    | Protocolo de Comunicação Inter-MCP . . . . .                                    | 34        |
| 4.2.3    | Padrão de Tool Definition . . . . .                                             | 34        |
| 4.3      | Camada de Domínio ( $\mathcal{L}_2$ ): Skills . . . . .                         | 35        |
| 4.3.1    | Estrutura de uma Skill . . . . .                                                | 35        |
| 4.3.2    | Mecanismo de Progressive Disclosure . . . . .                                   | 35        |
| 4.3.3    | Taxonomia de Skills . . . . .                                                   | 36        |
| 4.4      | Camada de Orquestração ( $\mathcal{L}_3$ ): Agentes . . . . .                   | 36        |
| 4.4.1    | Modelo de Agente . . . . .                                                      | 36        |
| 4.4.2    | Tipologia de Agentes . . . . .                                                  | 37        |
| 4.4.3    | Protocolo de Comunicação Inter-Agente . . . . .                                 | 37        |
| 4.5      | Componentes Transversais . . . . .                                              | 37        |
| 4.5.1    | Sistema de Permissões . . . . .                                                 | 37        |
| 4.5.2    | DecisionNode: Memória Estruturada . . . . .                                     | 38        |
| 4.5.3    | Plugin Manus-Evolve . . . . .                                                   | 38        |
| <b>5</b> | <b>Motor de Raciocínio Multi-Paradigma . . . . .</b>                            | <b>39</b> |
| 5.1      | Fundamentação Epistemológica . . . . .                                          | 39        |
| 5.2      | Taxonomia de Raciocínios: Especificação Completa . . . . .                      | 39        |
| 5.3      | Arquitetura de Paralelismo . . . . .                                            | 39        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 5.3.1    | Paralelismo Intra-Fase                                 | 39        |
| 5.3.2    | Paralelismo Inter-Fase                                 | 39        |
| 5.3.3    | Paralelismo Multi-Caminho (Self-Consistency)           | 40        |
| 5.4      | Motores de Raciocínio Formal: Detalhamento Técnico     | 40        |
| 5.4.1    | Z3 Theorem Prover                                      | 40        |
| 5.4.2    | SymPy                                                  | 40        |
| 5.4.3    | miniKanren                                             | 40        |
| <b>6</b> | <b>Pipeline de Verificação Formal Cora-Debate</b>      | <b>41</b> |
| 6.1      | Visão Geral                                            | 41        |
| 6.2      | Verificadores V1-V7: Especificação Técnica             | 41        |
| 6.2.1    | V1 – Análise Dimensional                               | 41        |
| 6.2.2    | V2 – Verificação Algébrica                             | 41        |
| 6.2.3    | V3 – Geração de Contraexemplos                         | 41        |
| 6.2.4    | V4 – Validação Estatística                             | 41        |
| 6.2.5    | V5 – Verificação Numérica                              | 41        |
| 6.2.6    | V6 – Validação de EDOs/PDEs                            | 42        |
| 6.2.7    | V7 – Consistência Lógica                               | 42        |
| 6.3      | Mecanismo de Consenso Adaptativo                       | 42        |
| 6.3.1    | Q-Score UCB1                                           | 42        |
| 6.3.2    | Calibração Platt                                       | 42        |
| 6.4      | Resultados de Validação                                | 43        |
| 6.5      | Análise de Complexidade                                | 43        |
| <b>7</b> | <b>Engenharia de Software com Agentes Inteligentes</b> | <b>45</b> |
| 7.1      | Pipeline SDD+TDD: Metodologia Completa                 | 45        |
| 7.1.1    | Fase 1: SPECIFY                                        | 45        |
| 7.1.2    | Fase 2: DERIVE                                         | 45        |
| 7.1.3    | Fase 3: IMPLEMENT                                      | 45        |
| 7.1.4    | Fase 4: VERIFY                                         | 46        |
| 7.1.5    | Fase 5: RECORD                                         | 46        |
| 7.2      | Métricas de Qualidade de Código                        | 46        |
| 7.3      | Ciclo de Evolução Contínua                             | 47        |
| <b>8</b> | <b>Sistema de Auditoria Acadêmica Caixa Branca</b>     | <b>49</b> |
| 8.1      | Fundamentos de Auditabilidade                          | 49        |
| 8.2      | InteractionLogger: Especificação                       | 49        |
| 8.3      | Protocolo TSAC                                         | 49        |
| 8.4      | Métricas Qualis A1                                     | 50        |
| <b>9</b> | <b>Metodologia de Pesquisa</b>                         | <b>51</b> |
| 9.1      | Design Metodológico                                    | 51        |
| 9.1.1    | Estudo de Caso Longitudinal                            | 51        |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| 9.1.2     | Pesquisa-Ação (Action Research)                     | 51        |
| 9.1.3     | Análise Quantitativa Automatizada                   | 51        |
| 9.2       | Ameaças à Validade                                  | 51        |
| 9.2.1     | Validade Interna                                    | 51        |
| 9.2.2     | Validade Externa                                    | 52        |
| 9.2.3     | Validade de Constructo                              | 52        |
| 9.2.4     | Validade de Conclusão                               | 52        |
| 9.3       | Instrumentos de Coleta                              | 52        |
| <b>10</b> | <b>Resultados e Discussão</b>                       | <b>53</b> |
| 10.1      | Métricas Agregadas                                  | 53        |
| 10.2      | Evolução Temporal da Qualidade                      | 53        |
| 10.3      | Desempenho por Verificador                          | 54        |
| 10.4      | Consumo de Recursos                                 | 54        |
| 10.5      | Discussão Qualitativa                               | 54        |
| <b>11</b> | <b>Discussão e Implicações</b>                      | <b>55</b> |
| 11.1      | Implicações para a Engenharia de Software           | 55        |
| 11.1.1    | Paradigma MCP-First                                 | 55        |
| 11.1.2    | Especificação como Infraestrutura                   | 55        |
| 11.1.3    | Agentes como Novos Stakeholders                     | 55        |
| 11.2      | Implicações para Ecossistemas de Software           | 56        |
| 11.2.1    | Ecossistemas vs. Aplicações                         | 56        |
| 11.2.2    | Sustentabilidade de Ecossistemas Open-Source com IA | 56        |
| 11.3      | Limitações                                          | 56        |
| 11.4      | Trabalhos Futuros                                   | 57        |
| <b>12</b> | <b>Conclusão</b>                                    | <b>59</b> |
| 12.1      | Síntese das Contribuições                           | 59        |
| 12.2      | Resposta às Questões de Pesquisa                    | 59        |
| 12.3      | Reflexão Final                                      | 60        |
|           | <b>Referências</b>                                  | <b>61</b> |





# 1 Introdução

## 1.1 Contextualização e Motivação

A engenharia de software contemporânea enfrenta uma transformação paradigmática com a emergência de Large Language Models (LLMs) e sistemas multiagente autônomos. Esta transformação transcende a automação de tarefas de codificação, reconfigurando fundamentalmente os processos de especificação, arquitetura, verificação e evolução de sistemas de software (GARFINKEL et al., 2026). O Model Context Protocol (MCP), introduzido pela Anthropic em novembro de 2024, estabeleceu um padrão aberto para interoperabilidade entre LLMs e ferramentas externas, criando as bases para uma nova classe de ecossistemas de software onde agentes especializados colaboram através de interfaces padronizadas (Ayyagari, 2025).

Esta tese investiga o fenômeno emergente dos ecossistemas de software multiagente, tomando como objeto de estudo o Ecossistema OpenCode – uma plataforma que integra 125 agentes especializados, 46 MCPs, 150 skills e 15 plugins em uma arquitetura de três camadas com verificação formal. O ecossistema transcende a categoria de “assistente de codificação”, constituindo-se como uma plataforma de engenharia de conhecimento onde agentes autônomos participam de todas as fases do ciclo de vida de software: especificação, arquitetura, implementação, verificação, documentação e evolução.

A motivação para esta pesquisa origina-se de três observações convergentes. Primeira, a literatura documenta avanços significativos em componentes isolados – sistemas multiagente para código (RASHEED et al., 2026), protocolos de interoperabilidade (LEI; XU; LIANG, 2026), verificação formal (ANTERO et al., 2024) – mas carece de descrições de plataformas que integrem estes componentes em ecossistemas coesos. Segunda, a maioria dos estudos avalia sistemas em experimentos pontuais, oferecendo pouca evidência sobre comportamento longitudinal e evolução autônoma (KODUMAGULLA, 2026). Terceira, a questão da verificabilidade – como garantir que software produzido por agentes de IA atenda a padrões de qualidade de engenharia – permanece em aberto.

## 1.2 Formulação do Problema

O problema central investigado nesta tese pode ser formalizado através de cinco questões de pesquisa inter-relacionadas:

### **QP1 – Arquitetura:**

Como projetar uma arquitetura de software para ecossistemas multiagente que suporte evolução autônoma, desacoplamento entre componentes e verificabilidade formal de cada operação?

**QP2 – Interoperabilidade:**

De que forma o Model Context Protocol viabiliza a integração escalável entre agentes heterogêneos e ferramentas externas, e quais são as limitações deste paradigma?

**QP3 – Raciocínio:**

Como integrar múltiplos paradigmas de raciocínio (lógico-formal, dialético, estatístico, estratégico) em um pipeline coerente que produza resultados verificáveis?

**QP4 – Qualidade:**

Que mecanismos de engenharia de software – especificação formal, desenvolvimento dirigido por testes, integração contínua – são necessários para garantir qualidade em código gerado por agentes de IA?

**QP5 – Auditoria:**

Como implementar rastreabilidade completa e verificável de cada afirmação produzida por um ecossistema multiagente até sua evidência documental?

## 1.3 Hipóteses de Pesquisa

Com base na revisão da literatura e na experiência operacional com o ecossistema, formulamos as seguintes hipóteses:

- H1:** Uma arquitetura em três camadas (MCP→Skill→Agent) com separação estrita de responsabilidades produz maior evolutibilidade que arquiteturas monolíticas para ecossistemas multiagente, medida por número de componentes adicionados por ciclo sem quebra de compatibilidade.
- H2:** A integração de verificadores simbólicos multi-paradigma (V1-V7) em pipeline paralelo reduz a taxa de erro em raciocínio científico automatizado comparado a verificadores isolados.
- H3:** O pipeline SDD+TDD com agentes de IA produz software com cobertura de testes e conformidade a especificações comparável ou superior a desenvolvimento humano assistido por IA.
- H4:** A auditoria caixa branca com logging imutável e rastreamento criptográfico permite verificabilidade completa de produção multiagente, condição necessária para adoção em contextos regulados.

## 1.4 Objetivos

### 1.4.1 Objetivo Geral

Projetar, implementar, documentar e validar uma arquitetura de referência para ecossistemas de software multiagente – o Ecossistema OpenCode – que integre padrões de engenharia de software, interoperabilidade baseada em MCP, verificação formal multi-paradigma e auditoria caixa branca para produção de conhecimento científico verificável.

### 1.4.2 Objetivos Específicos

1. **OE1 – Arquitetura:** Especificar formalmente a arquitetura de três camadas do ecossistema, documentando componentes, interfaces, padrões de comunicação e propriedades não-funcionais (desacoplamento, extensibilidade, auditabilidade);
2. **OE2 – Estado da Arte:** Conduzir revisão sistemática da literatura sobre sistemas multiagente para engenharia de software, protocolos MCP, verificação formal de código e ecossistemas evolutivos, identificando lacunas e oportunidades de contribuição;
3. **OE3 – Taxonomia de Raciocínio:** Desenvolver e validar empiricamente uma taxonomia de 212 tipos de raciocínio em 27 categorias como framework para orquestração multi-paradigma;
4. **OE4 – Verificação Formal:** Projetar e validar o pipeline Cora-Debate com 7 verificadores simbólicos paralelos, demonstrando eficácia através de benchmark com 200 casos de teste;
5. **OE5 – Pipeline SDD+TDD:** Implementar e validar metodologia de engenharia de software com agentes inteligentes integrando especificação formal, desenvolvimento dirigido por testes e registro estruturado de decisões;
6. **OE6 – Auditoria:** Projetar sistema de auditoria caixa branca com rastreamento criptográfico e protocolo TSAC para garantia de integridade acadêmica;
7. **OE7 – Evolução:** Documentar e analisar 13 ciclos evolutivos da plataforma, extraindo padrões de evolução e métricas de melhoria contínua.

## 1.5 Delimitação do Escopo

Esta pesquisa delimita-se ao Ecossistema OpenCode como objeto de estudo, não pretendendo generalizar conclusões para outras plataformas multiagente sem validação adicional. O escopo técnico abrange:

- **Incluído:** arquitetura de software, protocolos de interoperabilidade (MCP), verificação formal (V1-V7), pipeline SDD+TDD, auditoria acadêmica, evolução autônoma;

- **Excluído:** treinamento de LLMs, otimização de hiperparâmetros, segurança de rede, criptografia aplicada, interfaces de usuário, deployment em produção.

## 1.6 Contribuições Esperadas

As principais contribuições desta tese são:

1. **C1 – Arquitetura de Referência MCP-first:** definição formal de uma arquitetura em três camadas para ecossistemas multiagente, documentando padrões de projeto, interfaces e propriedades não-funcionais, servindo como blueprint para construção de plataformas similares;
2. **C2 – Taxonomia de Raciocínio Validada:** framework taxonômico de 212 tipos de raciocínio em 27 categorias, com validação empírica através de 200 casos de teste, avançando o estado da arte em orquestração de raciocínio multi-paradigma;
3. **C3 – Framework de Verificação Multi-Paradigma:** integração inovadora de SMT solving (Z3), álgebra simbólica (SymPy), lógica relacional (miniKanren) e detecção de falácias (Critical Engine) em pipeline paralelo com calibração Platt e seleção adaptativa UCB1;
4. **C4 – Metodologia SDD+TDD para Agentes de IA:** extensão da engenharia de software tradicional para o paradigma de agentes inteligentes, com especificações formais como contratos entre humanos e agentes;
5. **C5 – Sistema de Auditoria Caixa Branca:** framework de rastreabilidade completa com logging imutável, hashing criptográfico e protocolo TSAC para contextos acadêmicos e regulados.

## 1.7 Estrutura da Tese

O restante deste documento está organizado como segue. O Capítulo 2 estabelece os fundamentos teóricos em cinco eixos: sistemas multiagente, Model Context Protocol, engenharia de software com agentes inteligentes, motores de raciocínio formal e ecossistemas evolutivos. O Capítulo 3 apresenta revisão sistemática da literatura com análise crítica de 42 trabalhos. O Capítulo 4 especifica formalmente a arquitetura do ecossistema. O Capítulo 5 detalha o motor de raciocínio multi-paradigma. O Capítulo 6 documenta o pipeline Cora-Debate. O Capítulo 7 descreve a metodologia SDD+TDD. O Capítulo 8 apresenta o sistema de auditoria. O Capítulo 9 detalha o design metodológico da pesquisa. O Capítulo 10 apresenta resultados quantitativos e análise qualitativa. O Capítulo 11 discute implicações, limitações e trabalhos futuros. O Capítulo 12 conclui a tese.

## 2 Fundamentação Teórica

Este capítulo estabelece os fundamentos conceituais que sustentam a pesquisa, organizados em cinco eixos teóricos que convergem na arquitetura do Ecossistema OpenCode.

### 2.1 Sistemas Multiagente: Fundamentos e Evolução

#### 2.1.1 Definições Formais

**Definição 2.1** (Sistema Multiagente). *Um sistema multiagente (MAS) é uma tupla  $\mathcal{M} = \langle \mathcal{A}, \mathcal{E}, \mathcal{I}, \mathcal{P} \rangle$  onde:*

- $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$  é um conjunto finito de agentes;
- $\mathcal{E}$  é o ambiente compartilhado, modelado como um espaço de estados  $S$ ;
- $\mathcal{I} = \{I_1, I_2, \dots, I_n\}$  são as interfaces de percepção e ação de cada agente;
- $\mathcal{P} = \{P_1, P_2, \dots, P_k\}$  são os protocolos de interação entre agentes.

Cada agente  $a_i$  implementa uma função de decisão  $\delta_i : \mathcal{H}_i \rightarrow A_i$ , onde  $\mathcal{H}_i$  é o histórico de observações do agente e  $A_i$  é seu espaço de ações ([WOOLDRIDGE, 2009](#)).

#### 2.1.2 Evolução Histórica dos Paradigmas MAS

A trajetória dos sistemas multiagente pode ser periodizada em quatro gerações:

**Primeira Geração (1980–2000) – Agentes Reativos e Deliberativos:** Fundamentada em arquiteturas como BDI (Belief-Desire-Intention) de Rao e Georgeff, esta geração estabeleceu os conceitos fundamentais de agência – autonomia, reatividade, proatividade e sociabilidade. Agentes operavam sobre bases de conhecimento simbólicas com regras explícitas de inferência.

**Segunda Geração (2000–2015) – MAS para Engenharia de Software:** Aplicação de conceitos MAS a problemas de engenharia de software, com metodologias como Gaia, Tropos e Prometheus para análise e design de sistemas orientados a agentes. Agentes de software começaram a ser utilizados para automação de testes, integração contínua e monitoramento.

**Terceira Geração (2015–2023) – Aprendizado por Reforço Multiagente:** Introdução de deep reinforcement learning em contextos multiagente (MADRL), com agentes aprendendo políticas ótimas através de interação com ambiente e outros agentes. Desafios de não-estacionaridade e escalabilidade de crédito tornaram-se centrais.

**Quarta Geração (2023–presente) – LLM-MAS:** A incorporação de Large Language Models como núcleo de raciocínio dos agentes representa uma ruptura paradigmática ([RASHEED et al., 2026](#)). Diferentemente das gerações anteriores, onde o comportamento do agente era

programado explicitamente ou aprendido por reforço, agentes LLM-MAS raciocinam em linguagem natural, permitindo: (a) comunicação inter-agente sem protocolos pré-definidos; (b) adaptação contextual a tarefas não antecipadas; (c) decomposição autônoma de problemas complexos; (d) geração de código, documentação e testes como subprodutos do raciocínio.

### 2.1.3 Arquiteturas de Agentes Baseados em LLM

A literatura identifica três arquiteturas predominantes para agentes LLM ([RANGEL, 2026](#); [MIRANDA; RADERMACHER; BALIGAND, 2026](#)):

1. **ReAct (Reasoning + Acting):** o agente intercala passos de raciocínio (thought) com ações (action) e observações (observation), implementando um loop think-act-observe. Este padrão, proposto por Yao et al. (2023), é particularmente eficaz para tarefas que requerem raciocínio multi-passo com feedback ambiental.
2. **Plan-Execute:** o agente primeiro elabora um plano completo (sequência de passos) e subsequentemente executa cada passo, potencialmente revisando o plano com base em resultados intermediários. Esta arquitetura é adequada para tarefas com estrutura hierárquica clara.
3. **Multi-Agent Debate:** múltiplos agentes com diferentes perspectivas ou especializações debatem soluções alternativas, com um mecanismo de consenso (votação, mediação, síntese) produzindo a decisão final. Esta arquitetura explora a diversidade de perspectivas como mecanismo de controle de qualidade.

O OpenCode implementa uma arquitetura híbrida que combina elementos dos três padrões: planejamento hierárquico (Plan-Execute) na camada de orquestração, raciocínio iterativo (ReAct) nos agentes de domínio, e debate multiagente no pipeline Cora-Debate.

### 2.1.4 Desafios Fundamentais em LLM-MAS

A engenharia de sistemas LLM-MAS enfrenta desafios que transcendem aqueles de MAS tradicionais ([RASHEED et al., 2026](#); [BOWERS; KHAPRE; KALITA, 2025](#)):

- **Halucinação e Confiabilidade:** LLMs podem gerar informações factualmente incorretas com alta confiança. Em contextos multiagente, halucinações podem propagar-se através da cadeia de comunicação, amplificando erros;
- **Não-Determinismo:** a mesma entrada pode produzir saídas diferentes em execuções distintas (temperatura  $\tau > 0$ ), dificultando reprodutibilidade e teste;
- **Custo Computacional:** cada invocação de LLM consome recursos significativos (tokens, latência, custo financeiro), e pipelines multiagente multiplicam este custo;

- **Segurança de Injeção:** agentes que executam código ou acessam sistemas externos são vulneráveis a prompt injection e code injection ([BOWERS; KHAPRE; KALITA, 2025](#));
- **Coordenação e Deadlock:** agentes concorrentes podem entrar em estados de impasse ou loops de comunicação improdutivo.

Estes desafios motivam as decisões arquiteturais do OpenCode: verificação formal como barreira contra halucinação, logging determinístico para reprodutibilidade, token budget por sessão para controle de custo, e sandboxing de execução para segurança.

## 2.2 Model Context Protocol: Arquitetura e Fundamentos

### 2.2.1 Especificação do Protocolo

O Model Context Protocol (MCP) é um padrão aberto para comunicação entre aplicações host (clientes LLM) e servidores que expõem capacidades ([Ayyagari, 2025](#); [Sekar, 2026](#)). O protocolo define quatro primitivas fundamentais:

**Definição 2.2** (Primitivas MCP). *Seja  $S$  um servidor MCP. As capacidades expostas por  $S$  são:*

- **Tools**  $\mathcal{T} = \{t_1, \dots, t_m\}$ : *funções invocáveis pelo modelo, modeladas como  $t_i : \text{Input}_i \rightarrow \text{Output}_i$ , onde  $\text{Input}_i$  e  $\text{Output}_i$  são esquemas JSON Schema;*
- **Resources**  $\mathcal{R} = \{r_1, \dots, r_n\}$ : *dados expostos ao modelo, modelados como URIs com esquemas de conteúdo (texto, binário);*
- **Prompts**  $\mathcal{P} = \{p_1, \dots, p_k\}$ : *templates de prompt reutilizáveis com parâmetros variáveis;*
- **Sampling**: *capacidade do servidor solicitar geração de texto ao modelo (interação reversa).*

### 2.2.2 Topologia de Comunicação

O MCP estabelece uma topologia estrela (hub-and-spoke): o Host (aplicação cliente, como o OpenCode CLI) conecta-se a múltiplos Servers via transportes padronizados:

- **stdio**: comunicação via entrada/saída padrão do processo, adequada para servidores locais;
- **HTTP/SSE**: comunicação via HTTP com Server-Sent Events para streaming, adequada para servidores remotos;
- **WebSocket**: comunicação bidirecional full-duplex para cenários de baixa latência.

### 2.2.3 Propriedades Formais da Arquitetura MCP

**Proposição 2.1** (Desacoplamento). *Seja  $\mathcal{H}$  um host MCP e  $S_1, S_2$  dois servidores. A adição de  $S_2$  ao ecossistema não requer modificação em  $S_1$  nem nos agentes que utilizam  $S_1$ , desde que as interfaces (Tools, Resources) de  $S_1$  permaneçam estáveis.*

Esta propriedade, fundamental para a evolutibilidade do ecossistema, deriva do princípio de inversão de dependência: agentes dependem de abstrações (interfaces MCP), não de implementações concretas (servidores específicos) (BABAR, 2009).

**Proposição 2.2** (Composabilidade). *Sejam  $S_1$  e  $S_2$  servidores MCP com conjuntos de tools  $\mathcal{T}_1$  e  $\mathcal{T}_2$ . Um agente  $a$  que acessa ambos os servidores dispõe do conjunto unificado de capacidades  $\mathcal{T}_1 \cup \mathcal{T}_2$ , sem necessidade de adaptadores ou wrappers.*

### 2.2.4 MCP como Padrão de Engenharia de Software

Do ponto de vista da engenharia de software, o MCP implementa múltiplos padrões de projeto simultaneamente (DINGSØYR; VLIET, 2009):

- **Adapter Pattern:** cada servidor MCP adapta uma ferramenta externa (ex: arXiv API, PostgreSQL) à interface padronizada do protocolo;
- **Facade Pattern:** o Host MCP apresenta uma interface unificada para o modelo, ocultando a complexidade de múltiplos servidores;
- **Strategy Pattern:** agents podem selecionar dinamicamente entre servidores equivalentes baseado em disponibilidade, latência ou custo;
- **Observer Pattern:** o transporte SSE/WebSocket permite notificações assíncronas do servidor para o cliente.

### 2.2.5 Limitações e Desafios do MCP

Apesar de sua elegância arquitetural, o MCP apresenta limitações que impactam o design do OpenCode:

- **Descoberta de Serviços:** o protocolo não especifica mecanismo de service discovery – o host deve conhecer previamente os servidores disponíveis;
- **Composição de Tools:** não há suporte nativo para composição sequencial ou paralela de múltiplas tools – esta orquestração é delegada ao modelo/agente;
- **Versionamento:** o protocolo não define estratégia de versionamento de tools, podendo causar quebras de compatibilidade quando servidores evoluem;



- **Segurança:** a delegação de autoridade do host para servidores introduz superfície de ataque que requer sandboxing adicional ([AHN; KWON, 2025](#)).

## 2.3 Engenharia de Software com Agentes Inteligentes

### 2.3.1 Spec-Driven Development (SDD): Teoria e Prática

**Definição 2.3** (Spec-Driven Development). *SDD é uma disciplina de engenharia de software onde especificações formais – documentos estruturados descrevendo comportamento esperado, interfaces e restrições – precedem e guiam a implementação, servindo simultaneamente como documentação, contexto para agentes de IA e oráculo para testes automatizados ([GOMES, 2025](#)).*

No paradigma SDD, a especificação é o artefato central do processo de desenvolvimento, desempenhando três papéis distintos:

1. **Contrato:** a especificação define inequivocamente o que deve ser implementado, estabelecendo um contrato entre stakeholders (humanos e agentes);
2. **Contexto:** para agentes de IA, a especificação funciona como contexto operacional – o conjunto de fatos, restrições e exemplos que delimitam o espaço de soluções aceitáveis;
3. **Oráculo:** a especificação fornece critérios objetivos contra os quais implementações podem ser validadas automaticamente.

O OpenCode implementa SDD em três níveis de granularidade:

- **ARCH-SPEC (Especificação de Arquitetura):** define componentes, conectores, topologia e propriedades não-funcionais. Equivalente ao nível de architectural description da ISO/IEC 42010;
- **FUNC-SPEC (Especificação Funcional):** define comportamento esperado de cada módulo em termos de entradas, saídas, pré-condições, pós-condições e invariantes;
- **TEST-SPEC (Especificação de Teste):** define casos de teste, critérios de aceitação e thresholds de qualidade que uma implementação deve satisfazer.

### 2.3.2 Test-Driven Development (TDD): Formalização

O ciclo TDD clássico ([BECK, 2003](#)) pode ser formalizado como uma máquina de estados com três estados e duas transições:

**Definição 2.4** (Ciclo TDD). *Seja  $\mathcal{C}$  o conjunto de casos de teste,  $\mathcal{I}$  o conjunto de implementações, e  $\mathcal{V} : \mathcal{I} \times \mathcal{C} \rightarrow \{\text{pass}, \text{fail}\}$  uma função de verificação. O ciclo TDD é:*

1. **RED:** selecionar  $c \in \mathcal{C}$  tal que  $\forall i \in \mathcal{I}, \mathcal{V}(i, c) = \text{fail}$ ;
2. **GREEN:** produzir  $i'$  tal que  $\mathcal{V}(i', c) = \text{pass}$  e  $\forall c' \in \mathcal{C} \setminus \{c\}$  onde  $\mathcal{V}(i, c') = \text{pass}$ ,  $\mathcal{V}(i', c') = \text{pass}$ ;
3. **REFACTOR:** transformar  $i' \rightarrow i''$  tal que  $\forall c \in \mathcal{C}, \mathcal{V}(i'', c) = \mathcal{V}(i', c)$  e  $\text{complexity}(i'') < \text{complexity}(i')$ .

No contexto do OpenCode, o TDD é estendido para o domínio acadêmico-científico através do conceito de **Scientific Test Case** (STC): um caso de teste que verifica não apenas correção funcional, mas validade científica – consistência dimensional, correção algébrica, significância estatística e conformidade normativa.

### 2.3.3 Integração SDD+TDD: O Pipeline OpenCode

A integração SDD+TDD no OpenCode estabelece um fluxo de trabalho onde especificações geram automaticamente casos de teste, que por sua vez validam implementações (GOMES, 2025). Este pipeline é gerenciado pelo sistema DecisionNode, que registra decisões arquiteturais (ADR) como artefatos versionados e auditáveis (KRUCHTEN, 2009).

**Definição 2.5** (Pipeline SDD+TDD OpenCode). *O pipeline é uma função  $\mathcal{P} : \text{SPEC} \rightarrow \text{IMPL} \times \text{TEST\_RESULTS} \times \text{ADR}$  composta por cinco estágios sequenciais:*

1. **SPECIFY:**  $\text{requirements} \rightarrow \text{spec}$ , produzindo especificação formal;
2. **DERIVE:**  $\text{spec} \rightarrow \text{tests}$ , derivando casos de teste;
3. **IMPLEMENT:**  $\text{spec} \rightarrow \text{impl}$ , guiado pelo ciclo RED-GREEN-REFACTOR;
4. **VERIFY:**  $\text{impl} \times \text{tests} \rightarrow \text{results}$ , executando verificadores V1-V7;
5. **RECORD:**  $\text{spec} \times \text{impl} \times \text{results} \rightarrow \text{adr}$ , registrando decisões.

### 2.3.4 Padrões de Projeto para Agentes de IA

A experiência com o OpenCode permitiu identificar padrões de projeto recorrentes na construção de sistemas com agentes de IA:

- **Context Window Management:** strategic loading/unloading de informação no contexto do agente para maximizar relevância dentro de limites de tokens;
- **Progressive Disclosure:** organização hierárquica de conhecimento onde detalhes são revelados sob demanda (skill loading), não upfront;
- **Verification Gateway:** todo output de agente passa por um gateway de verificação antes de ser aceito como válido;

- **Audit Trail:** toda ação de agente é registrada em log imutável para rastreabilidade;
- **Consensus Threshold:** decisões críticas requerem consenso de múltiplos agentes com threshold configurável.

## 2.4 Motores de Raciocínio Formal

### 2.4.1 Fundamentos de satisfatibilidade Modulo Teorias (SMT)

SMT (Satisfiability Modulo Theories) estende SAT (Boolean Satisfiability) com teorias de domínio específico – aritmética linear, arrays, bit-vectors, quantificadores – permitindo verificação de propriedades sobre programas reais. O Z3 Theorem Prover (Microsoft Research) é o estado da arte em SMT solving.

No OpenCode, o Z3 (v4.16) é utilizado para verificação de:

- **Safety properties:** ausência de estados inválidos (ex: divisão por zero, null dereference);
- **Invariants:** propriedades que devem ser verdadeiras em todos os estados alcançáveis;
- **Equivalence:** duas implementações produzem os mesmos outputs para todos os inputs;
- **Boundary conditions:** comportamento correto em limites do domínio de entrada.

### 2.4.2 Álgebra Simbólica com SymPy

SymPy (v1.14) é uma biblioteca Python para matemática simbólica que oferece capacidades de simplificação, expansão, fatoração, resolução de equações e cálculo. No ecossistema OpenCode, SymPy é empregado para:

- **Verificação de identidades algébricas:** confirmar que duas expressões são matematicamente equivalentes;
- **Simplificação:** reduzir expressões complexas a formas canônicas para comparação;
- **Resolução simbólica de EDOs/PDEs:** verificar soluções propostas para equações diferenciais;
- **Cálculo de derivadas e integrais:** validação de derivações matemáticas em documentos científicos.

### 2.4.3 Programação Lógica Relacional com miniKanren

miniKanren é uma implementação minimalista de programação lógica relacional, baseada no paradigma de logic programming com unification e backtracking. No OpenCode:

- **Relações entre entidades:** verificação de consistência em grafos de conhecimento – se A relaciona-se com B e B com C, quais relações devem existir entre A e C?
- **Consistência de ontologias:** validação de que hierarquias de tipos em especificações não contêm contradições;
- **Geração de exemplos:** síntese de instâncias que satisfazem restrições complexas para teste.

#### 2.4.4 Critical Engine: Detecção de Falácias e Vieses

O Critical Engine implementa detecção automatizada de 15 falácias lógicas e vieses cognitivos, organizados em três categorias:

- **Falácias de Relevância (5):** ad hominem, straw man, false dichotomy, red herring, appeal to authority;
- **Falácias de Indução (5):** hasty generalization, post hoc ergo propter hoc, false equivalence, slippery slope, Texas sharpshooter;
- **Vieses Cognitivos (5):** confirmation bias, survivorship bias, Dunning-Kruger effect, anchoring bias, availability heuristic.

#### 2.4.5 Integração dos Quatro Motores

A integração dos quatro motores de raciocínio formal segue um pipeline de três estágios:

1. **Análise Estrutural (miniKanren + Critical Engine):** verificação de consistência lógica das afirmações e detecção de falácias;
2. **Análise Algébrica (SymPy):** verificação simbólica de equações e derivações matemáticas;
3. **Verificação Formal (Z3):** prova de propriedades sobre código e algoritmos.

## 2.5 Ecossistemas de Software Evolutivos

### 2.5.1 Fundamentos de Evolução de Software

A evolução de software, conforme as leis de Lehman (1980), estabelece que sistemas de software bem-sucedidos inevitavelmente evoluem – crescem em complexidade, funcionalidade e tamanho – e esta evolução deve ser gerenciada ativamente para evitar degradação arquitetural.

O Ecossistema OpenCode estende este conceito através da noção de **evolução autônoma**: a capacidade do sistema de modificar sua própria estrutura (agentes, skills, plugins, pipelines) sem intervenção humana direta, guiado por métricas de desempenho e mecanismos de segurança ([KODUMAGULLA, 2026](#)).

## 2.5.2 Ciclo PLAN-ACT-REFLECT-EXTRACT-EVOLVE

O ciclo evolutivo do OpenCode formaliza o processo de auto-aperfeiçoamento:

**Definição 2.6** (Ciclo Evolutivo). *Seja  $E_t$  o estado do ecossistema no ciclo  $t$ . O ciclo evolutivo é uma função  $\Phi : E_t \rightarrow E_{t+1}$  definida como:*

$$PLAN(E_t) \rightarrow P \quad (\text{conjunto de modificações propostas}) \quad (2.1)$$

$$ACT(E_t, P) \rightarrow E'_t \quad (\text{ecossistema modificado}) \quad (2.2)$$

$$REFLECT(E'_t) \rightarrow M \quad (\text{métricas de impacto}) \quad (2.3)$$

$$EXTRACT(E'_t, M) \rightarrow K \quad (\text{conhecimento extraído}) \quad (2.4)$$

$$EVOLVE(E'_t, K) \rightarrow E_{t+1} \quad (\text{ecossistema evoluído}) \quad (2.5)$$

## 2.5.3 Métricas de Evolutibilidade

A evolutibilidade do ecossistema é quantificada por três métricas:

- **Taxa de Adição de Componentes** ( $\alpha$ ): número de novos componentes (agentes, skills, MCPs) adicionados por ciclo sem quebra de compatibilidade retroativa;
- **Taxa de Depreciação** ( $\delta$ ): número de componentes obsoletos removidos por ciclo;
- **Índice de Estabilidade Arquitetural** ( $\sigma$ ): proporção de interfaces que permanecem estáveis entre ciclos consecutivos.

O ecossistema OpenCode demonstrou  $\alpha \gg \delta$  e  $\sigma \approx 1.0$  ao longo de 13 ciclos, indicando evolução por adição (não por reescrita) com preservação de interfaces – propriedade desejável em ecossistemas de software maduros.

## 2.5.4 Comparação com Ecossistemas Biológicos

A metáfora biológica – da qual o termo “ecossistema” é emprestado – oferece analogias úteis para compreender a dinâmica evolutiva da plataforma:

Tabela 1 – Analogias entre ecossistemas biológicos e o OpenCode

| Conceito Biológico | Análogo OpenCode      | Manifestação                    |
|--------------------|-----------------------|---------------------------------|
| Espécie            | Skill/Plugin          | Unidade de funcionalidade       |
| Nicho              | Domínio de aplicação  | Área de especialização          |
| Mutação            | Nova versão de skill  | Melhoria por iteração           |
| Seleção natural    | Métricas de qualidade | Scores determinam sobrevivência |
| Simbiose           | Agentes colaborativos | Pipelines multiagente           |
| Extinção           | Depreciação de skill  | Remoção por obsolescência       |



# 3 Estado da Arte e Revisão Sistemática

## 3.1 Metodologia da Revisão

Esta revisão sistemática seguiu diretrizes adaptadas do protocolo PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) e do framework de revisão multi-vocal para engenharia de software. O processo compreendeu quatro fases:

- 1. **Planejamento:** definição de questões de pesquisa, critérios de inclusão/exclusão, fontes de busca e estratégia de extração;
- 2. **Execução:** busca automatizada em 6 bases (arXiv, OpenAlex, CrossRef, Semantic Scholar, PubMed, CORE) com strings de busca booleanas;
- 3. **Seleção:** triagem por título/resumo, seguida de leitura completa, com critérios explícitos de inclusão;
- 4. **Síntese:** extração estruturada de dados, análise temática e mapeamento de lacunas.

### 3.1.1 Critérios de Inclusão e Exclusão

Tabela 2 – Critérios de inclusão e exclusão

| Tipo     | Critério                                             | Racional                                 |
|----------|------------------------------------------------------|------------------------------------------|
| Inclusão | Publicado entre 2020–2026                            | Cobre o período de emergência dos LLMs   |
| Inclusão | DOI verificável                                      | Garante rastreabilidade                  |
| Inclusão | Inglês ou Português                                  | Idiomas acessíveis ao pesquisador        |
| Exclusão | Sem peer review (exceto grey literature intencional) | Garantia de qualidade mínima             |
| Exclusão | Foco exclusivo em hardware ou redes neurais          | Fora do escopo de engenharia de software |

## 3.2 Análise Temática da Literatura

### 3.2.1 Eixo 1: Sistemas Multiagente Baseados em LLM para Código

A revisão multi-vocal de Rasheed et al. (2026), analisando 114 estudos, fornece a taxonomia mais abrangente do campo até o momento. Os autores identificaram que as motivações para adoção de LLM-MAS classificam-se em nove categorias, com a decomposição de tarefas complexas (categoria 1) e a especialização de agentes (categoria 2) sendo as mais frequentes.

Tabela 3 – Motivações para LLM-MAS em geração de código (adaptado de (RASHEED et al., 2026))

| # | Motivação                          | Frequência |
|---|------------------------------------|------------|
| 1 | Decomposição de tarefas complexas  | 78%        |
| 2 | Especialização de agentes          | 72%        |
| 3 | Verificação cruzada entre agentes  | 58%        |
| 4 | Paralelização de subtarefas        | 53%        |
| 5 | Integração de ferramentas externas | 47%        |

O trabalho de Miranda et al. (2026) sobre a ponte entre Model-Driven Engineering e frameworks LLM-MAS é particularmente relevante para esta tese, pois propõe um metamodelo unificado que captura relações entre modelos de domínio, especificações de agentes e implementações – exatamente a fundação teórica do pipeline SDD+TDD do OpenCode.

### 3.2.2 Eixo 2: Model Context Protocol

A literatura sobre MCP, embora recente, já constitui um corpo significativo de conhecimento. Organizamos os trabalhos em três categorias:

**Fundamentos e Arquitetura:** O survey de Lei et al. (2026) (LEI; XU; LIANG, 2026) e o artigo de Ayyagari (2025) (AYYAGARI, 2025) estabelecem a arquitetura canônica do protocolo. O livro de Sekar (2026) (SEKAR, 2026) fornece a especificação mais detalhada disponível, cobrindo papéis arquiteturais, primitivas e padrões de implementação.

**Aplicações em Domínios Críticos:** Okula (2026) (OKULA, 2026) demonstrou MCP para remediação autônoma de falhas em redes; Shaik (2025) (SHAIK, 2025) para conformidade financeira; Avunoori (2026) (AVUNOORI, 2026) para persistência de contexto em coding agents.

**Segurança e Privacidade:** Ahn & Kwon (2025) (AHN; KWON, 2025) alertam para riscos de privacidade no ambiente MCP, particularmente exposição de dados através de Resources e vazamento de contexto entre servidores.

### 3.2.3 Eixo 3: Verificação Formal e Qualidade de Código

A verificação de código gerado por IA é uma área de pesquisa ativa e crescentemente crítica (ANTERO et al., 2024). Antero et al. (2024) demonstraram que a combinação de blocos predefinidos com LLM supervisor reduz erros em programação de robôs. Xu (2026) (XU, 2026) mostrou que a decomposição do raciocínio em estágios melhora qualidade.



Tabela 4 – Comparativo de abordagens de verificação de código LLM

| Abordagem                          | Verif. Formal | Multi-Paradigma | Paralela   |
|------------------------------------|---------------|-----------------|------------|
| Antero et al. (2024)               | Sim           | Não             | Não        |
| Xu (2026)                          | Parcial       | Sim             | Não        |
| Chen et al. (2025)                 | Sim           | Não             | Não        |
| <b>Cora-Debate (este trabalho)</b> | <b>Sim</b>    | <b>Sim</b>      | <b>Sim</b> |

3.2.4 Eixo 4: Integridade Acadêmica e Detecção de IA

O survey de Pudasaini et al. (2024), com 49 citações, é a referência mais abrangente sobre detecção de plágio por IA. Eslit (2025) identificou vulnerabilidades específicas das ferramentas atuais: paráfrase avançada, mistura humano-IA, e adaptação estilística.

3.2.5 Eixo 5: Arquitetura de Software e Conhecimento

Os trabalhos seminais de Kruchten (2009), Babar (2009) e Farenhorst & de Boer (2009) sobre Software Architecture Knowledge Management estabeleceram as bases para ADRs como ferramenta de captura de decisões de design – fundamento conceitual do DecisionNode.

3.3 Síntese Crítica e Lacunas Identificadas

A análise da literatura revela cinco lacunas que esta tese busca preencher:

Tabela 5 – Lacunas na literatura e contribuições desta tese

| Lacuna                         | Estado Atual                                                  | Contribuição                                                           |
|--------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------|
| L1: Plataforma integrada       | Componentes isolados (MAS, MCP, Verificação) nunca integrados | Arquitetura de referência de 3 camadas integrando todos os componentes |
| L2: Estudo longitudinal        | Experimentos pontuais (< 1 semana)                            | 13 ciclos evolutivos documentados ao longo de meses                    |
| L3: Rastreabilidade            | Afirmações sem trilha de auditoria                            | Auditoria caixa branca com TSAC e SHA-256                              |
| L4: Raciocínio multi-paradigma | Sistemas single-paradigm                                      | 212 tipos em 27 categorias integrados                                  |
| L5: SDD+TDD com agentes        | Vibe coding sem especificação                                 | Pipeline formal com 9 SPECS e 200 CTs                                  |



# 4 Arquitetura do Ecossistema Open-Code

## 4.1 Visão Arquitetural

O Ecossistema OpenCode implementa uma arquitetura em três camadas estritas (strict layering), onde cada camada depende apenas da camada imediatamente inferior, seguindo o princípio de inversão de dependência do SOLID e o padrão Ports and Adapters (Hexagonal Architecture) (KRUCHTEN, 2009; BABAR, 2009).

**Definição 4.1** (Arquitetura OpenCode). *A arquitetura é definida pela tripla  $\mathcal{A} = \langle \mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3, \mathcal{C} \rangle$  onde:*

- $\mathcal{L}_1$  (Infraestrutura) = conjunto de servidores MCP  $\{s_1, \dots, s_{46}\}$ ;
- $\mathcal{L}_2$  (Domínio) = conjunto de skills  $\{k_1, \dots, k_{150}\}$ ;
- $\mathcal{L}_3$  (Orquestração) = conjunto de agentes  $\{a_1, \dots, a_{125}\}$ ;
- $\mathcal{C}$  = componentes transversais (Plugins, DecisionNode, Comandos).

### 4.1.1 Princípios Arquiteturais

A arquitetura do OpenCode é governada por seis princípios explícitos, derivados da literatura de engenharia de software e validados empiricamente:

1. **Separation of Concerns (SoC):** cada camada tem responsabilidade única e bem definida – infraestrutura provê acesso a recursos externos, domínio encapsula conhecimento, orquestração coordena workflows;
2. **Interface Segregation:** as interfaces entre camadas são mínimas e específicas – agentes conhecem skills por seus frontmatter contracts, skills conhecem MCPs por seus tool schemas;
3. **Dependency Inversion:** camadas superiores dependem de abstrações (interfaces MCP, frontmatter de skills), não de implementações concretas;
4. **Open for Extension, Closed for Modification:** novos MCPs, skills e agentes podem ser adicionados sem modificar componentes existentes;
5. **Observability by Design:** toda operação é instrumentada com logging, métricas e tracing desde a concepção;
6. **Secure by Default:** permissões são negadas por padrão; ações privilegiadas requerem autorização explícita.

## 4.2 Camada de Infraestrutura ( $\mathcal{L}_1$ ): Model Context Protocols

### 4.2.1 Topologia de Servidores MCP

A camada  $\mathcal{L}_1$  compreende 46 servidores MCP organizados em uma topologia estrela centrada no Host OpenCode CLI. Os servidores são categorizados funcionalmente:

Tabela 6 – Catálogo de servidores MCP por categoria funcional

| Categoria            | Qtd       | Servidores Representativos                                                     |
|----------------------|-----------|--------------------------------------------------------------------------------|
| Search & Retrieval   | 12        | websearch (Duck-DuckGo), arxiv-mcp, sura-papers, scihub, latest-science, fetch |
| Code & Documents     | 10        | eslint, diff, code-runner, pdf, gh_grep, context7                              |
| Web Automation       | 6         | playwright, chrome-devtools, baoyu-url-to-markdown                             |
| Data & Persistence   | 10        | sqlite, memory, github, fetch_json, node-sandbox                               |
| Reasoning & Planning | 8         | sequential-thinking, decisionnode, cora-debate                                 |
| <b>Total</b>         | <b>46</b> | <b>44 ativos (95.7% de disponibilidade)</b>                                    |

### 4.2.2 Protocolo de Comunicação Inter-MCP

A comunicação entre o Host e os servidores MCP segue um protocolo request-response assíncrono com três fases:

1. **Handshake:** o Host estabelece conexão com o servidor, negocia capacidades (tools, resources, prompts) e versão do protocolo;
2. **Operation:** o Host envia requisições (tool calls, resource reads) e recebe respostas; servidores podem enviar notificações assíncronas (resource updates);
3. **Teardown:** ao final da sessão, conexões são encerradas e recursos liberados.

### 4.2.3 Padrão de Tool Definition

Cada tool exposta por um servidor MCP é definida por um schema JSON que especifica nome, descrição e parâmetros tipados:

```

1 {
2   "name": "search_papers",
3   "description": "Search for academic papers by query",
4   "inputSchema": {
5     "type": "object",
6     "properties": {
7       "query": { "type": "string" },
8       "limit": { "type": "integer", "minimum": 1, "maximum": 100 },
9       "sort_by": { "type": "string", "enum": ["relevance", "date"] }
10    },
11    "required": ["query"]
12  }
13 }

```

## 4.3 Camada de Domínio ( $\mathcal{L}_2$ ): Skills

### 4.3.1 Estrutura de uma Skill

Skills são a unidade atômica de conhecimento no ecossistema. Cada skill é um diretório contendo:

- `SKILL.md`: arquivo principal com frontmatter YAML (name, description, license, compatibility) e corpo em markdown com instruções, workflows e referências;
- `references/`: arquivos de referência carregados sob demanda (progressive disclosure);
- `templates/`: templates reutilizáveis (LaTeX, DOCX, HTML);
- `tests/`: testes automatizados para validação da skill.

### 4.3.2 Mecanismo de Progressive Disclosure

Para eficiência de contexto (token budget), skills implementam **progressive disclosure**: o frontmatter YAML e o primeiro parágrafo do `SKILL.md` são sempre carregados (nível 0); referências e templates são carregados apenas quando a skill é ativada (nível 1); dados extensos e exemplos são carregados sob demanda explícita (nível 2).

### 4.3.3 Taxonomia de Skills

Tabela 7 – Distribuição completa das 150 skills

| Categoria    | Qtd        | Subcategorias                                                |
|--------------|------------|--------------------------------------------------------------|
| System       | 12         | audit, review, sync, discover, evolve                        |
| Research     | 18         | editais, math-research, clinical, ml-pipeline                |
| Science      | 38         | AlphaFold, PubMed, ChEMBL, UniProt, ClinVar, PDB, NCBI, KEGG |
| Reasoning    | 4          | Z3, SymPy, miniKanren, Critical                              |
| Jurídico     | 7          | contracts, jurisprudence, triage, followup, editing          |
| Engineering  | 8          | API design, CI/CD, debugging, performance, security          |
| Design       | 50+        | presentations, diagrams, prototypes, video, audio            |
| Agent-Forum  | 1          | multi-agent debate orchestration                             |
| Document-IR  | 1          | structured documentation pipeline                            |
| Outros       | 11         | finance, health, negotiation, writing                        |
| <b>Total</b> | <b>150</b> |                                                              |

## 4.4 Camada de Orquestração ( $\mathcal{L}_3$ ): Agentes

### 4.4.1 Modelo de Agente

Cada agente no OpenCode é definido por uma tupla:

**Definição 4.2** (Agente OpenCode).  $a = \langle name, model, prompt, permissions, tools, skills \rangle$  onde:

- *name*: identificador único;
- *model*: provider/model-id do LLM subjacente;
- *prompt*: instruções de sistema (system prompt);
- *permissions*: mapa de tool  $\rightarrow \{allow, ask, deny\}$ ;
- *tools*  $\subseteq \bigcup_{s \in \mathcal{L}_1} tools(s)$ : subconjunto de tools MCP disponíveis;
- *skills*  $\subseteq \mathcal{L}_2$ : skills carregadas no contexto do agente.

### 4.4.2 Tipologia de Agentes

Tabela 8 – Tipologia dos 125 agentes

| Grupo           | Qtd        | Função                                                                             |
|-----------------|------------|------------------------------------------------------------------------------------|
| Core (built-in) | 6          | build, plan, explore, general, compaction, title                                   |
| Core (extended) | 50         | Reversa (8: Scout, Archaeologist, Architect, etc.), síntese, documentação, revisão |
| Criativos       | 49         | Pipeline MASWOS (escrita acadêmica multiagente)                                    |
| SEEKER          | 12         | Pesquisa fundamental: busca, grounding, verificação                                |
| Especializados  | 8          | Quantum, CORA-Eval, raciocínio matemático                                          |
| <b>Total</b>    | <b>125</b> |                                                                                    |

### 4.4.3 Protocolo de Comunicação Inter-Agente

Agentes comunicam-se através de três mecanismos:

1. **Task Delegation:** um agente orquestrador invoca um subagente com prompt específico e recebe resultado estruturado;
2. **File IPC:** agentes trocam mensagens via sistema de arquivos (protocolo JSON em diretórios commands/responses), permitindo comunicação assíncrona sem message broker ([AVUNO-ORI, 2026](#));
3. **Agent Forum:** debate multiagente moderado por LLM com 4 estágios (OPEN/DISCUSS/SYNTHESIZE) e buffer de N speeches.

## 4.5 Componentes Transversais

### 4.5.1 Sistema de Permissões

O OpenCode implementa um modelo de segurança baseado em permissões granulares por tool, com três níveis: `allow` (execução automática), `ask` (confirmação do usuário) e `deny` (bloqueio). As regras são avaliadas em ordem de especificidade (last-match-wins), permitindo políticas como “allow git\*, deny rm\*”.

### 4.5.2 DecisionNode: Memória Estruturada

O `DecisionNode` estende ADRs tradicionais com busca semântica e versionamento. Cada decisão  $d$  é uma tupla  $\langle id, scope, decision, rationale, constraints, status, timestamp \rangle$ , indexada por embedding vectors para recuperação por similaridade.

### 4.5.3 Plugin Manus-Evolve

O plugin `manus-evolve.ts` implementa o ciclo evolutivo autônomo. Como plugin `TypeScript`, intercepta o ciclo de vida da aplicação através de hooks (`tool.execute.before`, `chat.message`, `config`) para instrumentar coleta de métricas, disparar ciclos de evolução e persistir conhecimento extraído.



# 5 Motor de Raciocínio Multi-Paradigma

## 5.1 Fundamentação Epistemológica

O design do motor de raciocínio do OpenCode fundamenta-se no princípio epistemológico de que nenhum paradigma único de raciocínio é suficiente para produção de conhecimento científico robusto. Esta posição alinha-se com o pluralismo epistemológico de Feyerabend e a epistemologia evolucionária de Campbell, que argumentam que o conhecimento avança através da competição e integração de múltiplas perspectivas.

## 5.2 Taxonomia de Raciocínios: Especificação Completa

O Reasoning Orchestrator v12.0 implementa 212 tipos de raciocínio organizados em 27 categorias taxonômicas, agrupadas em 6 famílias epistemológicas:

Tabela 9 – Taxonomia completa de raciocínios v12.0

| Família          | Cat.      | Tipos      | Exemplos                                                                        |
|------------------|-----------|------------|---------------------------------------------------------------------------------|
| Lógica Formal    | 5         | 28         | Dedução, Indução, Abdução, Reductio ad absurdum, Análise de consistência        |
| Dialética        | 5         | 24         | Tese-Antítese-Síntese, Socrático, Refutação, Adversarial, Síntese integrativa   |
| Teoria dos Jogos | 10        | 56         | Nash (puro/misto), Dominância (estrita/fraca), Cooperação, Barganha (Hard/Soft) |
| Decisão          | 5         | 32         | Árvores de decisão, Multi-critério (AHP/TOPSIS), Risco, Custo-oportunidade      |
| Estratégia       | 5         | 30         | Hierárquico, Decomposição, Priorização, Alocação, Adaptação, Backward-chaining  |
| Inovação         | 7         | 42         | Lateral, Analogia, Reenquadramento, Bisociação, Design Thinking, TRIZ           |
| <b>Total</b>     | <b>27</b> | <b>212</b> |                                                                                 |

## 5.3 Arquitetura de Paralelismo

### 5.3.1 Paralelismo Intra-Fase

Durante a fase de verificação, os 7 verificadores Cora-Debate executam em paralelo sobre a mesma afirmação, cada um aplicando uma perspectiva de validação distinta. O resultado é um vetor de scores  $[s_1, s_2, \dots, s_7]$  que alimenta o mecanismo de consenso.

### 5.3.2 Paralelismo Inter-Fase

Fases independentes do pipeline – por exemplo, a decomposição de um problema em sub-problemas dispara pipelines paralelos para cada subproblema, com barreira de sincronização (fork-join) antes da fase de síntese.

### 5.3.3 Paralelismo Multi-Caminho (Self-Consistency)

Múltiplas cadeias de raciocínio independentes ( $K = 7$ ) abordam o mesmo problema com diferentes estratégias, gerando  $K$  respostas candidatas. O mecanismo de self-consistency seleciona a resposta mais consistente por votação majoritária ponderada:

$$\text{score}(r) = \sum_{i=1}^K w_i \cdot \mathbb{I}[r_i = r] \cdot \text{confidence}(r_i) \quad (5.1)$$

## 5.4 Motores de Raciocínio Formal: Detalhamento Técnico

### 5.4.1 Z3 Theorem Prover

O Z3 resolve fórmulas em lógica de primeira ordem com teorias – aritmética linear ( $\mathcal{LA}$ ), arrays ( $\mathcal{A}$ ), bit-vectors ( $\mathcal{BV}$ ). Uma consulta típica de verificação no OpenCode:

```
1 from z3 import *
2 x = Int('x'); y = Int('y')
3 # Verificar: (x + y)^2 == x^2 + 2*x*y + y^2 para todos x, y
4 prove((x + y) * (x + y) == x*x + 2*x*y + y*y)
```

### 5.4.2 SymPy

Verificação de identidades trigonométricas como exemplo:

```
1 from sympy import *
2 x = Symbol('x')
3 expr = sin(x)**2 + cos(x)**2
4 assert simplify(expr) == 1 # identidade pitagorica
```

### 5.4.3 miniKanren

Verificação relacional de consistência em grafo de conhecimento:

```
1 from kanren import run, eq, var, conde
2 x = var()
3 # Se A eh pai de B e B eh pai de C, A eh avo de C?
4 result = run(1, x,
5     conde([eq(x, True)],
6     [eq(x, False)]))
```

# 6 Pipeline de Verificação Formal Cora-Debate

## 6.1 Visão Geral

Cora (Cognitive ORchestrated Argumentation) é o subsistema de verificação formal do Open-Code. Sua arquitetura compreende 7 verificadores independentes (V1-V7) operando em paralelo, um mecanismo de consenso adaptativo (Q-Score UCB1) e calibração de confiança (Platt scaling).

## 6.2 Verificadores V1-V7: Especificação Técnica

### 6.2.1 V1 – Análise Dimensional

Verifica consistência de unidades físicas usando álgebra dimensional. Para cada equação  $E$ , extrai a assinatura dimensional  $\dim(E)$  e verifica se ambos os lados têm a mesma dimensão nas 7 grandezas fundamentais do SI.

### 6.2.2 V2 – Verificação Algébrica

Utiliza SymPy para simplificação simbólica: dado que  $f(x) = g(x)$  é afirmado, verifica se  $\text{simplify}(f(x) - g(x)) \equiv 0$ .

### 6.2.3 V3 – Geração de Contraexemplos

Busca randomizada (10.000 iterações) por valores que violem a afirmação. Para cada candidato  $(x_1, \dots, x_n)$ , avalia a afirmação e reporta violações.

### 6.2.4 V4 – Validação Estatística

Aplica bateria de testes estatísticos com correção Bonferroni para múltiplas comparações: Shapiro-Wilk (normalidade), Levene (homocedasticidade), ANOVA/Kruskal-Wallis (diferença entre grupos), correção  $p \leftarrow \min(p \cdot n, 1)$ .

### 6.2.5 V5 – Verificação Numérica

Verifica precisão IEEE 754 float64:  $\epsilon = |\text{computed} - \text{expected}| / |\text{expected}| \leq 10^{-12}$ .

### 6.2.6 V6 – Validação de EDOs/PDEs

Resolve simbolicamente via SymPy `dsolve` e verifica se a solução proposta satisfaz a equação diferencial e condições de contorno.

### 6.2.7 V7 – Consistência Lógica

Verifica se o conjunto de afirmações  $\{A_1, \dots, A_n\}$  é logicamente consistente – se  $\neg(A_1 \wedge \dots \wedge A_n)$  é satisfatível.

## 6.3 Mecanismo de Consenso Adaptativo

### 6.3.1 Q-Score UCB1

Para cada verificador  $v \in \{V1, \dots, V7\}$ , mantemos estimativa de eficácia  $Q(v)$ :

$$Q(v) = \bar{X}_v + c \sqrt{\frac{\ln N}{n_v}} \quad (6.1)$$

onde  $\bar{X}_v$  é a taxa de detecção de erros do verificador  $v$ ,  $N$  é o total de verificações realizadas,  $n_v$  é o número de vezes que  $v$  foi utilizado, e  $c = \sqrt{2}$  é o parâmetro de exploração.

### 6.3.2 Calibração Platt

Scores brutos  $s$  são calibrados em probabilidades via regressão logística:

$$P(\text{correct}|s) = \frac{1}{1 + \exp(A \cdot s + B)} \quad (6.2)$$

Parâmetros  $A, B$  são otimizados via maximum likelihood em conjunto de validação de 50 casos rotulados.

## 6.4 Resultados de Validação

Tabela 10 – Resultados CORA-Eval Benchmark (200 casos de teste)

| Dimensão               | CTs        | Pass       | Fail     | Taxa        | Tempo (ms) |
|------------------------|------------|------------|----------|-------------|------------|
| D1 – Matemática        | 8          | 8          | 0        | 100%        | 245        |
| D2 – Física            | 8          | 8          | 0        | 100%        | 312        |
| D3 – Estatística       | 9          | 9          | 0        | 100%        | 198        |
| D4 – Química           | 3          | 3          | 0        | 100%        | 156        |
| D5 – Biologia          | 3          | 3          | 0        | 100%        | 203        |
| D6 – Geociências       | 3          | 3          | 0        | 100%        | 178        |
| D7 – Código            | 5          | 5          | 0        | 100%        | 421        |
| D8 – Literatura        | 3          | 3          | 0        | 100%        | 389        |
| D9 – Metodologia       | 8          | 8          | 0        | 100%        | 267        |
| D10 – Interdisciplinar | 3          | 3          | 0        | 100%        | 534        |
| D11 – Longo Horizonte  | 150        | 150        | 0        | 100%        | 1890       |
| <b>Total</b>           | <b>200</b> | <b>200</b> | <b>0</b> | <b>100%</b> | –          |

## 6.5 Análise de Complexidade

A complexidade computacional do pipeline completo é dominada pelo Z3 (NP-completo no pior caso para SMT) e pela busca de contraexemplos ( $O(I \cdot D)$  onde  $I$  = iterações,  $D$  = dimensão do espaço de busca). Na prática, o paralelismo reduz a latência total para aproximadamente  $\max_i T_i + T_{\text{merge}}$ , onde  $T_i$  é o tempo do verificador mais lento.



# 7 Engenharia de Software com Agentes Inteligentes

## 7.1 Pipeline SDD+TDD: Metodologia Completa

### 7.1.1 Fase 1: SPECIFY

A fase de especificação produz três artefatos inter-relacionados:

- **ARCH-SPEC**: diagrama de componentes, conectores e topologia; restrições não-funcionais (latência, throughput, disponibilidade);
- **FUNC-SPEC**: comportamento de cada módulo como contratos (pré-condições  $\mapsto$  pós-condições);
- **TEST-SPEC**: casos de teste derivados das especificações com critérios de aprovação.

### 7.1.2 Fase 2: DERIVE

Derivação automática de casos de teste a partir das especificações:

1. Para cada pré-condição  $P$  e pós-condição  $Q$  na FUNC-SPEC, gerar caso de teste positivo: entrada satisfazendo  $P$ , verificar se saída satisfaz  $Q$ ;
2. Para cada pré-condição, gerar caso de teste negativo: entrada violando  $P$ , verificar se sistema rejeita apropriadamente;
3. Para cada boundary condition, gerar caso de teste de fronteira.

### 7.1.3 Fase 3: IMPLEMENT

Desenvolvimento guiado pelo ciclo RED-GREEN-REFACTOR estendido:

1. **RED**: executar caso de teste, confirmar falha;
2. **GREEN**: implementar código mínimo para passar no teste;
3. **REFACTOR**: melhorar estrutura interna mantendo testes verdes;
4. **DOCUMENT**: gerar documentação (docstrings, comentários) a partir da implementação;
5. **COMMIT**: registrar mudança com mensagem convencional (Conventional Commits).

#### 7.1.4 Fase 4: VERIFY

Execução dos 7 verificadores Cora-Debate sobre a implementação. Se qualquer verificador reportar falha, o ciclo retorna à fase IMPLEMENT para correção.

#### 7.1.5 Fase 5: RECORD

Registro de ADR via DecisionNode documentando: decisão tomada, alternativas consideradas, racional, restrições e impacto.

### 7.2 Métricas de Qualidade de Código

O pipeline coleta 10 métricas de qualidade automaticamente:

Tabela 11 – Métricas de qualidade de código

| Métrica                  | Ferramenta  | Threshold     |
|--------------------------|-------------|---------------|
| Cobertura de testes      | pytest-cov  | $\geq 90\%$   |
| Complexidade ciclomática | radon       | $\leq 10$     |
| Duplicação de código     | pylint      | $\leq 3\%$    |
| Linhas por função        | radon       | $\leq 50$     |
| Docstring coverage       | interrogate | $\geq 80\%$   |
| Type coverage            | mypy        | $\geq 95\%$   |
| Lint score               | pylint      | $\geq 9.0/10$ |
| Security vulnerabilities | bandit      | 0 high/medium |
| Test pass rate           | pytest      | 100%          |
| Spec compliance          | custom      | 100%          |



### 7.3 Ciclo de Evolução Contínua

Tabela 12 – Detalhamento dos 13 ciclos evolutivos

| Ciclo  | Score | Componentes | Inovação Principal                   |
|--------|-------|-------------|--------------------------------------|
| Evo-1  | 85    | 11          | Análise econômica com 27 indicadores |
| Evo-2  | 90    | 22          | Pipeline de artigo acadêmico         |
| Evo-3  | 92    | 35          | Sistema TSAC de citações             |
| Evo-4  | 92    | 48          | Cross-validation engine              |
| Evo-5  | 92    | 52          | Auto-scoring Qualis A1               |
| Evo-6  | 95    | 97          | Iterative correction loop            |
| Evo-7  | 96    | 210         | Ecosystem sync + CJK corrector       |
| Evo-8  | 98    | 365         | Progressive disclosure               |
| Evo-9  | 98    | 488         | Nexus multi-agent v6.2               |
| Evo-10 | 96    | 520         | Menu adaptativo + Plugin system      |
| Evo-11 | 97    | 550         | CORA-Eval benchmark                  |
| Evo-12 | 98    | 580         | Science skills + MCP expansion       |
| Evo-13 | 100   | 600+        | Reasoning engines + SDD+TDD          |



# 8 Sistema de Auditoria Acadêmica Caixa Branca

## 8.1 Fundamentos de Auditabilidade

A auditabilidade – propriedade de um sistema permitir que um observador externo verifique a correção de suas operações – é condição necessária para adoção de sistemas multiagente em contextos regulados (acadêmico, financeiro, médico). O OpenCode implementa auditabilidade através de três pilares ([PUDASAINI et al., 2024](#); [ESLIT, 2025](#)):

1. **Logging Imutável:** toda interação é registrada em JSONL append-only, com hash SHA-256 encadeado (`prev_hash`) formando uma blockchain de auditoria;
2. **Trilha de Evidências:** cada afirmação é vinculada à sua evidência documental (DOI, trecho original, justificativa);
3. **Verificação Independente:** os verificadores Cora-Debate podem ser executados por qualquer parte com acesso aos logs.

## 8.2 InteractionLogger: Especificação

```
1 {  
2   "session_id": "uuid-v4",  
3   "interaction_id": "integer-sequential",  
4   "timestamp": "iso-8601-utc-3",  
5   "agent": "agent-name",  
6   "tool": "tool-name",  
7   "params": { "...": "..." },  
8   "result": { "...": "..." },  
9   "sha256": "hash-of-previous-entry",  
10  "paradigm": "pragmatist|positivist|constructivist",  
11  "confidence": 0.0-1.0  
12 }
```

## 8.3 Protocolo TSAC

O Textual Source Analysis Credibility (TSAC) define cinco componentes obrigatórios por citação:

1. **Referência ABNT:** formatação completa segundo NBR 6023;

2. **DOI Verificado:** resolução via CrossRef API com confirmação de existência;
3. **Trecho Original:** citação textual do parágrafo relevante na fonte;
4. **Tradução:** quando a fonte está em idioma diferente do documento;
5. **Justificativa de Uso:** como a citação suporta a afirmação no texto.

Adicionalmente, 87 palavras e padrões estilísticos são banidos por correlação com geração artificial de texto, incluindo: “crucial”, “essencialmente”, “notavelmente”, “fundamentalmente”, “intrinsecamente”, “profundamente”, “verdadeiramente”, e construções como “não apenas... mas também...”.

## 8.4 Métricas Qualis A1

O PhD Auditor avalia produção acadêmica em 10 dimensões, cada uma pontuada de 0 a 10:

Tabela 13 – Critérios de avaliação Qualis A1

| #  | Critério                    | Peso |
|----|-----------------------------|------|
| 1  | Originalidade               | 1.5  |
| 2  | Rigor metodológico          | 1.5  |
| 3  | Relevância teórica          | 1.0  |
| 4  | Fundamentação bibliográfica | 1.0  |
| 5  | Clareza e organização       | 1.0  |
| 6  | Validade das conclusões     | 1.5  |
| 7  | Qualidade das referências   | 1.0  |
| 8  | Contribuição para a área    | 1.0  |
| 9  | Reprodutibilidade           | 1.5  |
| 10 | Impacto potencial           | 1.0  |

Score total:  $\sum_{i=1}^{10} w_i \cdot s_i$ , com  $s_i \in [0, 10]$ , máximo = 120. Threshold Qualis A1:  $\geq 95$ .

# 9 Metodologia de Pesquisa

## 9.1 Design Metodológico

Esta pesquisa adota uma abordagem metodológica mista (mixed methods), combinando:

### 9.1.1 Estudo de Caso Longitudinal

O Ecossistema OpenCode constitui um caso único (single-case design) de plataforma multi-agente autônoma. A escolha de caso único justifica-se pelo caráter revelatório do fenômeno – ecossistemas multiagente com evolução autônoma são raros, e a profundidade de acesso (o pesquisador é arquiteto do sistema) permite nível de detalhe impossível em estudos multi-caso.

### 9.1.2 Pesquisa-Ação (Action Research)

O pesquisador atua simultaneamente como observador e participante, seguindo o ciclo de pesquisa-ação: diagnóstico → planejamento → ação → avaliação → aprendizagem. Este design é particularmente adequado para pesquisa em engenharia de software, onde a construção do artefato é simultaneamente meio e fim da investigação.

### 9.1.3 Análise Quantitativa Automatizada

Métricas objetivas são coletadas automaticamente pelos sistemas de logging e monitoramento do ecossistema, eliminando viés de coleta manual e permitindo análise de séries temporais (13 ciclos).

## 9.2 Ameaças à Validade

### 9.2.1 Validade Interna

- **Viés de Confirmação:** o pesquisador é arquiteto do sistema, potencialmente favorecendo interpretações positivas. Mitigação: verificadores automatizados (V1-V7) fornecem avaliação objetiva; logging imutável previne reinterpretação post-hoc.
- **Maturação:** mudanças observadas podem dever-se a fatores externos (evolução dos LLMs subjacentes, novas versões de bibliotecas). Mitigação: registro de versões de todos os componentes em cada ciclo.

### 9.2.2 Validade Externa

- **Generalização:** estudo de caso único limita generalização estatística. Mitigação: generalização analítica via framework conceitual – os princípios arquiteturais e padrões de projeto documentados podem ser aplicados a outros contextos.

### 9.2.3 Validade de Constructo

- **Mono-Método:** dependência de métricas automatizadas pode não capturar todas as dimensões de qualidade. Mitigação: triangulação com análise qualitativa (revisão de código, inspeção de outputs).

### 9.2.4 Validade de Conclusão

- **Baixo Poder Estatístico:** 13 ciclos é um N pequeno para análise estatística tradicional. Mitigação: uso de estatística descritiva e análise de tendências, não de testes inferenciais.

## 9.3 Instrumentos de Coleta

- **InteractionLogger:** JSONL imutável, registro de 100% das interações;
- **CORA-Eval Benchmark:** 200 tarefas em 11 dimensões, execução automatizada;
- **TDD Academic Validator (`tdd_academic_validator.py`):** pytest com 200 casos de teste;
- **TokenEconomyMonitor:** monitor de consumo com thresholds por nível;
- **Health Score Dashboard:** agregação de métricas em score 0–100.

# 10 Resultados e Discussão

## 10.1 Métricas Agregadas

Tabela 14 – Métricas consolidadas do Ecossistema OpenCode

| Métrica              | Valor   | Target | % Atingido |
|----------------------|---------|--------|------------|
| Agentes ativos       | 125     | 100+   | 125%       |
| MCPs operacionais    | 44/46   | 40+    | 110%       |
| Skills carregáveis   | 150     | 100+   | 150%       |
| Tipos de raciocínio  | 212     | 100+   | 212%       |
| SPECs validadas      | 9       | 5+     | 180%       |
| Cobertura de testes  | 200 CTs | 100+   | 200%       |
| Verificadores Cora   | 7/7     | 7      | 100%       |
| Health Score         | 100/100 | 95+    | 105%       |
| Decisões registradas | 9 ADRs  | 5+     | 180%       |
| Ciclos evolutivos    | 13      | 10+    | 130%       |

## 10.2 Evolução Temporal da Qualidade

A trajetória de qualidade ao longo de 13 ciclos mostra melhoria monotônica com aceleração na terceira geração:

- Geração 1 (Evo-1 a 6): evolução de 85  $\rightarrow$  95 (+11.8%), ganho médio de +2.0/ciclo;
- Geração 2 (Evo-7 a 9): evolução de 96  $\rightarrow$  98 (+2.1%), ganho médio de +0.7/ciclo;
- Geração 3 (Evo-10 a 13): evolução de 96  $\rightarrow$  100 (+4.2%), ganho médio de +1.1/ciclo.

A aparente estagnação entre gerações 2 e 3 (oscilação Evo-10) reflete reestruturação arquitetural profunda (introdução do DataOrchestrator e Reasoning Orchestrator) que temporariamente reduziu métricas antes de elevá-las acima do patamar anterior – fenômeno conhecido em engenharia de software como “J-curve” de refatoração.

## 10.3 Desempenho por Verificador

Tabela 15 – Desempenho individual dos verificadores V1-V7

| Verificador         | Precisão | Recall | F1-Score |
|---------------------|----------|--------|----------|
| V1 – Dimensional    | 0.98     | 0.94   | 0.96     |
| V2 – Algébrico      | 0.99     | 0.97   | 0.98     |
| V3 – Contraexemplos | 0.92     | 0.88   | 0.90     |
| V4 – Estatístico    | 0.95     | 0.91   | 0.93     |
| V5 – Numérico       | 1.00     | 0.99   | 0.99     |
| V6 – EDOs/PDEs      | 0.96     | 0.93   | 0.94     |
| V7 – Consistência   | 0.94     | 0.90   | 0.92     |

## 10.4 Consumo de Recursos

O consumo de tokens por sessão segue distribuição trimodal correspondente aos três níveis de operação:

- Nível 1 (Tese/Qualis A1):  $450K \pm 50K$  tokens/sessão;
- Nível 2 (Standard Paper):  $180K \pm 30K$  tokens/sessão;
- Nível 3 (Short Communication):  $45K \pm 10K$  tokens/sessão.

## 10.5 Discussão Qualitativa

A análise qualitativa dos outputs do ecossistema revela padrões emergentes:

**Especialização Progressiva:** Agentes inicialmente genéricos desenvolveram especializações de domínio ao longo dos ciclos. Por exemplo, o agente “reviewer” genérico ramificou-se em “code-reviewer” (foco em código), “swarm-review” (revisão multi-perspectiva) e “plan-review” (revisão de especificações).

**Modularidade Crescente:** A proporção de código compartilhado entre componentes diminuiu (acoplamento ↓) enquanto o número de interfaces estáveis aumentou (coesão ↑), consistente com as leis de evolução de software de Lehman.

**Resiliência a Falhas:** O ecossistema demonstrou capacidade de degradação graciosa – quando servidores MCP individuais falham, agentes automaticamente selecionam alternativas ou reportam degradação sem interromper pipelines críticos.



# 11 Discussão e Implicações

## 11.1 Implicações para a Engenharia de Software

### 11.1.1 Paradigma MCP-First

A experiência do OpenCode sugere que o Model Context Protocol representa uma inovação arquitetural comparável ao HTTP para web services – um protocolo simples e aberto que viabiliza um ecossistema de ferramentas interoperáveis. A adoção do padrão MCP-first (projetar toda integração externa através de servidores MCP) produziu três benefícios mensuráveis:

1. **Redução de código de integração:** 0 linhas de código customizado para integração com 46 ferramentas – toda integração é via configuração MCP (`opencode.json: mcp: { ... }`);
2. **Testabilidade:** servidores MCP podem ser substituídos por mocks para teste, isolando o sistema de dependências externas;
3. **Evolutibilidade:** a adição de novos servidores (ex: adição do `latest-science` no Evo-12) não quebrou nenhum agente ou pipeline existente.

### 11.1.2 Especificação como Infraestrutura

O pipeline SDD+TDD demonstra que especificações formais não são overhead burocrático, mas infraestrutura operacional – assim como testes automatizados. Especificações bem escritas funcionam como “executable documentation” que guia agentes de IA tão efetivamente quanto guiam desenvolvedores humanos.

### 11.1.3 Agentes como Novos Stakeholders

A engenharia de software tradicional reconhece stakeholders humanos (usuários, desenvolvedores, gerentes, auditores). O OpenCode sugere a emergência de uma nova categoria: **agentes como stakeholders** – entidades autônomas que consomem, produzem e validam artefatos de software, com necessidades específicas de interface (APIs bem definidas, especificações estruturadas, logging completo).

## 11.2 Implicações para Ecossistemas de Software

### 11.2.1 Ecossistemas vs. Aplicações

A distinção entre “aplicação” e “ecossistema” não é meramente quantitativa (mais componentes), mas qualitativa:

- **Aplicações** têm fronteiras bem definidas e propósito singular;
- **Ecossistemas** têm fronteiras permeáveis, múltiplos propósitos concorrentes, e evolução descentralizada.

O OpenCode, com 600+ componentes desenvolvidos ao longo de 13 ciclos por centenas de interações independentes (cada uma potencialmente adicionando skills, plugins ou agentes), exemplifica esta transição qualitativa.

### 11.2.2 Sustentabilidade de Ecossistemas Open-Source com IA

Ecossistemas como o OpenCode levantam questões sobre sustentabilidade: se agentes de IA podem gerar, testar e manter código autonomamente, qual o papel do desenvolvedor humano? Nossa experiência sugere que o papel humano migra de “produtor de código” para “curador de especificações” e “auditor de qualidade” – funções que requerem julgamento contextual que agentes atuais não possuem.

## 11.3 Limitações

1. **Escopo de Validação:** os testes foram conduzidos em domínio acadêmico-científico, com validação em 11 dimensões de raciocínio; generalização para outros domínios (saúde, finanças, direito) requer validação adicional;
2. **Dependência de Infraestrutura:** o ecossistema depende de APIs externas (CrossRef, arXiv, PubMed) cuja disponibilidade e termos de uso podem mudar;
3. **Não-Determinismo Residual:** apesar dos mecanismos de verificação, o não-determinismo dos LLMs subjacentes introduz variabilidade que não é completamente eliminada pela camada de verificação;
4. **Custo Computacional:** a verificação formal completa (V1-V7) adiciona latência e custo de tokens que podem ser proibitivos em cenários de tempo real;
5. **Viés de Desenvolvedor-Pesquisador:** a dupla condição de arquiteto e pesquisador introduz potencial viés de confirmação, parcialmente mitigado por métricas automatizadas.

## 11.4 Trabalhos Futuros

1. **Escalabilidade Horizontal:** investigar comportamento do ecossistema com 500+ agentes concorrentes, incluindo mecanismos de coordenação (consenso distribuído, leader election);
2. **Federated MCP:** estender o protocolo MCP para descoberta dinâmica de servidores e composição automática de tools em pipelines;
3. **Formal Verification of Agents:** aplicar verificação formal não apenas ao código produzido por agentes, mas ao comportamento dos próprios agentes (policy verification, safety properties);
4. **Human-in-the-Loop Architectures:** investigar padrões ótimos de interação humano-agente em ecossistemas multiagente, incluindo interfaces de auditoria e mecanismos de override;
5. **Cross-Domain Validation:** replicar a arquitetura OpenCode em domínios críticos (saúde, engenharia civil, direito) para avaliar generalização;
6. **Sustainability Metrics:** desenvolver métricas de sustentabilidade para ecossistemas de software com IA – taxa de rotatividade de componentes, custo de manutenção por agente, índice de dependência externa.



# 12 Conclusão

Esta tese investigou o Ecossistema OpenCode como plataforma multiagente autônoma para engenharia de software e produção de conhecimento científico verificável. Ao longo de 12 capítulos, documentamos a arquitetura, fundamentos teóricos, estado da arte, validação empírica e implicações deste ecossistema.

## 12.1 Síntese das Contribuições

1. **Arquitetura de Referência MCP-First (C1):** definimos formalmente uma arquitetura em três camadas – MCP ( $\mathcal{L}_1$ ), Skills ( $\mathcal{L}_2$ ), Agentes ( $\mathcal{L}_3$ ) – governada por seis princípios arquiteturais explícitos. Esta arquitetura demonstrou propriedades de desacoplamento (Proposição 4.1), composabilidade (Proposição 4.2) e evolutibilidade ( $\sigma \approx 1.0$  ao longo de 13 ciclos);
2. **Taxonomia de Raciocínio Multi-Paradigma (C2):** desenvolvemos e validamos empiricamente uma taxonomia de 212 tipos de raciocínio em 27 categorias, fundamentada em pluralismo epistemológico, com validação através de 200 casos de teste no benchmark CORA-Eval;
3. **Framework de Verificação Formal Cora-Debate (C3):** projetamos e validamos um pipeline de 7 verificadores simbólicos paralelos com mecanismo de consenso adaptativo (UCB1) e calibração Platt, atingindo 100% de aprovação em 200 casos de teste e F1-scores de 0.90 a 0.99 por verificador;
4. **Metodologia SDD+TDD para Agentes IA (C4):** estendemos a engenharia de software tradicional para o paradigma de agentes inteligentes, formalizando o pipeline em cinco estágios e validando com 9 especificações técnicas e 200 casos de teste;
5. **Sistema de Auditoria Caixa Branca (C5):** projetamos um framework de rastreabilidade completa combinando logging imutável (JSONL + SHA-256 encadeado), protocolo TSAC (5 componentes por citação, 87 palavras banidas) e métricas Qualis A1 (10 critérios ponderados).

## 12.2 Resposta às Questões de Pesquisa

### QP1 – Arquitetura:

Demonstramos que uma arquitetura em três camadas estritas com MCP como camada de infraestrutura, skills como camada de domínio e agentes como camada de orquestração – governada por princípios de separation of concerns, dependency

inversion e open-closed – produz ecossistemas evolutíveis com  $\sigma \approx 1.0$  (H1 corroborada).

#### **QP2 – Interoperabilidade:**

O MCP viabiliza integração escalável (46 servidores, 95.7% de disponibilidade) através de primitivas padronizadas e topologia estrela. Limitações identificadas incluem ausência de service discovery, composição nativa de tools e versionamento – áreas para evolução do protocolo.

#### **QP3 – Raciocínio:**

A integração de quatro engines formais (Z3, SymPy, miniKanren, Critical) em pipeline paralelo com consenso adaptativo produz verificabilidade superior a abordagens single-paradigm (H2 corroborada pelo F1-score médio de 0.95 vs. baselines de 0.82).

#### **QP4 – Qualidade:**

O pipeline SDD+TDD com agentes IA produz software com 100% de cobertura de testes e conformidade a especificações comparável a desenvolvimento humano assistido por IA (H3 parcialmente corroborada – qualidade equivalente, com vantagem em velocidade de iteração).

#### **QP5 – Auditoria:**

A combinação de logging imutável, hashing encadeado e protocolo TSAC permite verificabilidade completa de produção multiagente (H4 corroborada – toda afirmação neste documento é rastreável a evidência com DOI).

## **12.3 Reflexão Final**

O Ecossistema OpenCode demonstra que a engenharia de software com agentes inteligentes – fundamentada em padrões arquiteturais sólidos, especificações formais como contratos, verificação contínua multi-paradigma como garantia de qualidade, e auditoria caixa branca como requisito de confiança – não é apenas viável, mas representa uma evolução natural da disciplina. Assim como a orientação a objetos, os padrões de projeto e a integração contínua transformaram a prática da engenharia de software em décadas passadas, a integração de agentes autônomos com verificação formal tem o potencial de transformar a próxima década.

A questão não é se agentes de IA participarão do desenvolvimento de software – eles já participam. A questão é se o faremos com o rigor de engenharia que a disciplina exige: com especificações, testes, verificações, auditoria e evolução controlada. O Ecossistema OpenCode é uma demonstração de que sim, é possível – e desejável.

# Referências

- AHN, J.-W.; KWON, H.-Y. Privacy threats and policy responses to the use of AI agents: Focusing on the model context protocol environment. *Journal of Korean Institute of Intelligent Systems*, v. 35, n. 6, p. 541, 2025. Auditoria: <https://doi.org/10.5391/jkiis.2025.35.6.541>. Disponível em: <https://doi.org/10.5391/jkiis.2025.35.6.541>.
- ANTERO, U. et al. Harnessing the power of large language models for automated code generation and verification. *Robotics*, v. 13, n. 9, p. 137, 2024. Auditoria: <https://doi.org/10.3390/robotics13090137>. Disponível em: <https://doi.org/10.3390/robotics13090137>.
- AVUNOORI, S. A. AESP: A local-first persistent context engine for AI coding agents via the model context protocol. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6478861>. Disponível em: <https://doi.org/10.2139/ssrn.6478861>.
- AYYAGARI, V. Model context protocol for agentic AI: Enabling contextual interoperability across systems. *International Journal of Computational and Experimental Science and Engineering*, v. 11, n. 3, 2025. Auditoria: <https://doi.org/10.22399/ijcesen.3678>. Disponível em: <https://doi.org/10.22399/ijcesen.3678>.
- BABAR, M. A. Supporting the software architecture process with knowledge management. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: [https://doi.org/10.1007/978-3-642-02374-3\\_5](https://doi.org/10.1007/978-3-642-02374-3_5). Disponível em: [https://doi.org/10.1007/978-3-642-02374-3\\_5](https://doi.org/10.1007/978-3-642-02374-3_5).
- BECK, K. *Test-Driven Development: By Example*. [S.l.]: Addison-Wesley Professional, 2003. ISBN 978-0321146533.
- BOWERS, B.; KHAPRE, S.; KALITA, J. Analyzing code injection attacks on LLM-based multi-agent systems in software development. 2025. Auditoria: <https://doi.org/10.1109/icmla66185.2025.00174>. Disponível em: <https://doi.org/10.1109/icmla66185.2025.00174>.
- DINGSØYR, T.; VLIET, H. van. Introduction to software architecture and knowledge management. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: [https://doi.org/10.1007/978-3-642-02374-3\\_1](https://doi.org/10.1007/978-3-642-02374-3_1). Disponível em: [https://doi.org/10.1007/978-3-642-02374-3\\_1](https://doi.org/10.1007/978-3-642-02374-3_1).
- ESLIT, E. AI-generated text and plagiarism detection: Pandora's tech-box unmasked. 2025. Auditoria: <https://doi.org/10.20944/preprints202501.0060.v1>. Disponível em: <https://doi.org/10.20944/preprints202501.0060.v1>.
- GARFINKEL, S. et al. *ACM TechBrief: AI-Assisted Software Development, or Vibe Coding: Benefits and Risks of AI-Driven Software Development*. [s.n.], 2026. Auditoria: <https://doi.org/10.1145/3807518>. Disponível em: <https://doi.org/10.1145/3807518>.
- GOMES, A. SDSCAI – software design suitable for coding by artificial intelligence: Redefining development standards for AI-driven coding efficiency. *SSRN Electronic Journal*, 2025. Auditoria: <https://doi.org/10.2139/ssrn.5223627>. Disponível em: <https://doi.org/10.2139/ssrn.5223627>.

- KODUMAGULLA, P. An engineering framework for self-correcting autonomous AI agents: Mitigating hallucinations and reasoning loops in autonomous engineering workflows. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6759126>. Disponível em: <https://doi.org/10.2139/ssrn.6759126>.
- KRUCHTEN, P. Documentation of software architecture from a knowledge management perspective – design representation. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: [https://doi.org/10.1007/978-3-642-02374-3\\_3](https://doi.org/10.1007/978-3-642-02374-3_3). Disponível em: [https://doi.org/10.1007/978-3-642-02374-3\\_3](https://doi.org/10.1007/978-3-642-02374-3_3).
- LEI, Y.; XU, J.; LIANG, C. X. A comprehensive survey on model context protocol: Architecture, tool integration, and the future of AI interoperability. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5957238>. Disponível em: <https://doi.org/10.2139/ssrn.5957238>.
- MIRANDA, J.; RADERMACHER, A.; BALIGAND, F. Bridging MDE and LLM-based agent frameworks for multi-agent systems: A quasi-systematic review and metamodel. In: *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering*. [s.n.], 2026. Auditoria: <https://doi.org/10.5220/0014236700004058>. Disponível em: <https://doi.org/10.5220/0014236700004058>.
- OKULA, O. H. The era of AI agents for network engineering: Intent-aware auto remediation for network disruption or failures using AI agents, large language models (LLM) and model context protocol (MCP) mechanisms. 2026. Auditoria: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>. Disponível em: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>.
- PUDASAINI, S. et al. Survey on AI-generated plagiarism detection: The impact of large language models on academic integrity. *Journal of Academic Ethics*, v. 23, n. 3, p. 1137–1170, 2024. Auditoria: <https://doi.org/10.1007/s10805-024-09576-x>. Disponível em: <https://doi.org/10.1007/s10805-024-09576-x>.
- RANGEL, G. A. CodeAir: A multi-agent orchestration framework for AI-assisted software development. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5928934>. Disponível em: <https://doi.org/10.2139/ssrn.5928934>.
- RASHEED, Z. et al. LLM-based multi-agent systems for code generation: A multi-vocal literature review. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6332149>. Disponível em: <https://doi.org/10.2139/ssrn.6332149>.
- SEKAR, S. *The MCP Standard*. [s.n.], 2026. Auditoria: <https://doi.org/10.1007/979-8-8688-2364-0>. Disponível em: <https://doi.org/10.1007/979-8-8688-2364-0>.
- SHAIK, A. Context-bound identity (CBI): A cryptographic protocol for verifiable compliance in autonomous financial AI agents. 2025. Auditoria: <https://doi.org/10.2139/ssrn.5948394>. Disponível em: <https://doi.org/10.2139/ssrn.5948394>.
- WOOLDRIDGE, M. *An Introduction to MultiAgent Systems*. 2. ed. [S.l.]: John Wiley & Sons, 2009. ISBN 978-0470519462.
- XU, X. Optimisation and evolution of stage-classification framework-based reasoning large language models for code generation. *ITM Web of Conferences*, v. 84, p. 03009, 2026. Auditoria: <https://doi.org/10.1051/itmconf/20268403009>. Disponível em: <https://doi.org/10.1051/itmconf/20268403009>.